

Fase 1: Diagnóstico y Auditoría de Calidad de Datos para la Detección de Degradación en Registros Clínicos

Nombres integrantes:

Gonzalo Bouldres V

Luis Díaz G

Eduardo Contreras C

Curso: 202681.2535 Programación para la Ciencia de Datos

I. Índice

I. Índice	2
Capítulo I Introducción y Contextualización.....	5
1.1. Contextualización de la Ingesta de Datos Clínicos	6
1.2. Marco Teórico I: Teoría de la Degradación y Entropía de Datos.....	7
1.3. Marco Teórico II: El Paradigma de la Reproducibilidad Científica	9
1.4. Justificación Técnica del Enfoque Python	10
1.5. Matriz de Conexión Metodológica F1–F4	12
Capítulo II Definición de la Problemática y Objetivos del Proyecto	16
2.1. Problemática Delimitada	17
2.2. Pregunta o Necesidad Técnica que Guía el Análisis.....	18
2.3. Objetivo General	19
2.4. Objetivos Específicos Medibles	20
2.5. Alcance y Limitaciones	21
2.5.1. Fronteras del Alcance (Dentro de Alcance).....	21
2.5.2. Fronteras del Alcance (Fuera de Alcance)	21
2.5.3. Limitaciones Tecnológicas y Operacionales	22
2.6. Supuestos: Calidad del Dataset y Variables	22
Capítulo III Aplicación de Herramientas Científicas	24
3.1. El Ecosistema Científico de Python	25
3.4. Integración de Jupyter con el Repositorio Base	29
3.4.1. Protocolo de Control de Versiones para Cuadernos Científicos	29
3.4.2. Cumplimiento de la Idempotencia bajo el Estándar <i>Restart & Run All</i>	31
Capítulo IV Reproducibilidad Técnica.....	32
4.1. Ambiente Virtual e Infraestructura de Aislamiento	33

4.2. Dependencias Realmente Usadas	33
4.3. Instrucciones de Despliegue y Comandos Base de Reproducibilidad	34
4.4. Validación de Ejecución del Notebook sin Errores	34
Capítulo V Control de Versiones.....	36
5.1 Repositorio	37
5.2 Estructura del Proyecto	37
5.2.1 Descripción de carpetas:	37
5.3 Contributors:	38
5.4 Tabla de prefijos definidos para el proyecto.....	39
5.5. Ejemplo de versionamiento	40
5.6 Ejemplo de Ramas	41
Capítulo VI Notebook F1_Definicion.ipynb	42
6.1 Estructura del Notebook	43
6.2 Modularidad	44
6.2 Trazabilidad	45
6.3 Reproducibilidad del Notebook	45
Capítulo VII Documentación del proceso	46
7.1 Decisiones técnicas de la fase	47
7.2 Justificación de cambios: motivo e impacto	49
7.2.1 Enfoques evaluados y descartados	51
7.3 Coherencia entre informe, notebook y repositorio	51
Capítulo VIII Repositorio GitHub correspondiente a la Fase 1	53
8.1 Estructura del repositorio según el mapa conceptual	54
8.1.1 Arquitectura General del Repositorio	54
8.1.2 Estructura Interna por Fase	55
8.2 README.md preliminar.....	57

8.3 Commits informativos con trazabilidad	57
8.4 Contribuciones individuales preliminares	59
Capítulo IX Vinculación del mapa conceptual con el avance	60
9.1 Correspondencia entre el mapa conceptual y lo implementado en F1	62
9.2 Elementos implementados	63
9.3 Elementos proyectados para fases posteriores	64
9.4 Cierre de coherencia técnica	65
Capítulo X Bibliografía	66



Capítulo I

Introducción y Contextualización

1.1. Contextualización de la Ingesta de Datos Clínicos

La digitalización de los sistemas de salud a nivel global ha impulsado la adopción masiva de los Registros Clínicos Electrónicos (EHR, por sus siglas en inglés), transformando la gobernanza, el almacenamiento y la explotación de la información médica (World Health Organization [WHO], 2023). En la actualidad, el desarrollo de la medicina de precisión y la implementación de sistemas inteligentes de soporte a la decisión clínica dependen directamente de la capacidad técnica para transformar flujos masivos de datos crudos (*Raw Data*) en evidencia científica e inferencial totalmente libre de sesgos (Topol, 2021). Sin embargo, el proceso crítico de ingesta de datos en entornos hospitalarios reales se enfrenta a un desafío estructural adverso: la captura descentralizada de información, la heterogeneidad de los sistemas de bases de datos y la manipulación manual de registros clínicos en tiempo real (Rajkomar et al., 2019). Esta desconexión operativa inyecta altos niveles de entropía informacional en los repositorios institucionales, manifestándose en forma de ruido estadístico, inconsistencias sintácticas y severas pérdidas de información que degradan de forma sistemática la calidad de los datos antes de su procesamiento analítico.

Como escenario de estudio metodológico orientado a caracterizar y mitigar estas patologías de infraestructura mediante el enfoque de Aprendizaje Basado en Problemas (ABP), este proyecto aborda la ingesta del archivo de datos crudos denominado `diabetes_raw.csv`. La matriz de datos bajo análisis presenta una volumetría inicial concentrada en un total de 2045 registros indexados de forma transversal. El esquema nominal del dataset está constituido por 9 variables analíticas, diseñadas para capturar atributos clínicos, demográficos y fisiológicos fundamentales de los pacientes: `Pregnancies`, `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, `BMI`, `DiabetesPedigreeFunction`, `Age` y el indicador categórico dicotómico de diagnóstico `Outcome`.

A pesar del valor predictivo potencial subyacente en los atributos del set de datos, el diagnóstico estático inicial ejecutado en el entorno de ejecución científico revela de inmediato la presencia de anomalías estructurales severas causadas por una captura deficiente en el punto de origen. En primer lugar, se identifican importantes vacíos de información cuantitativa que comprometen la representatividad de la muestra, registrándose un vector de pérdidas absolutas equivalente a 164 valores faltantes en la variable `SkinThickness` y 245 omisiones críticas en el indicador metabólico de `Insulin`. En segundo lugar, la variable que mide la presión arterial (`BloodPressure`) ha sufrido una degradación total de su tipo de dato nativo, quedando parseada por el motor de Pandas como una variable cualitativa de tipo `object`; esto confirma la presencia de cadenas de texto parásitas o caracteres de escape alfanuméricos mezclados con la métrica numérica original. Finalmente, el análisis de distribución empírica del vector objetivo `Outcome` exhibe un desbalance de clases controlado, donde el 64.79% de las observaciones corresponden a pacientes sanos (Clase 0) frente a un 35.21% de diagnósticos positivos de diabetes (Clase 1).

Este escenario inicial fundamenta la relevancia técnica del problema analítico abordado. La presencia de datos faltantes, duplicados, valores atípicos y variables con tipos no esperados limita la

•

aplicación directa de modelos predictivos, ya que puede afectar la estabilidad del entrenamiento, la interpretación estadística y la calidad de los resultados obtenidos. Por consiguiente, la justificación técnica de esta primera fase no radica en modificar los registros, sino en establecer una infraestructura reproducible de auditoría y diagnóstico inicial. Este entorno actúa como línea base para estructurar posteriormente el pipeline de limpieza y transformación lógica (ETL), garantizando que las decisiones de las fases siguientes se fundamenten en evidencia informacional trazable y validada.

1.2. Marco Teórico I: Teoría de la Degradación y Entropía de Datos

El diseño de un pipeline de Ingeniería de Datos dentro del paradigma de la ciencia de datos moderna exige una fundamentación teórica rigurosa sobre cómo la información se corrompe durante sus fases de captura, transmisión y almacenamiento. Lejos de ser un fenómeno meramente operativo o circunstancial, la pérdida de calidad en los datos crudos (*Raw Data*) puede modelarse formalmente mediante la “Teoría de la Información y la entropía de Shannon”¹ (1948). Bajo este enfoque, cualquier sistema de recopilación de registros clínicos distribuidos se comporta como un **canal de comunicación ruidoso**, donde la señal original (el estado fisiológico real del paciente) es distorsionada por fuentes de ruido estocástico² e interferencias operacionales antes de consolidarse en la base de datos de destino. La entropía informacional de un conjunto de datos actúa, por tanto, como una métrica de la incertidumbre, el desorden estructural y la pérdida de pureza matemática de las variables analíticas. A mayor entropía en la ingesta, menor es la capacidad de los modelos de aprendizaje automático para extraer patrones probabilísticos verdaderos, lo que introduce sesgos algorítmicos críticos y reduce la estabilidad del espacio de características.

¹ La **Teoría de la Información**, desarrollada por Claude Shannon en 1948, es el marco matemático que estudia la cuantificación, almacenamiento y comunicación de la información. Su objetivo principal es optimizar la transmisión de datos y minimizar el impacto del ruido.

² Se refiere a un proceso o modelo que incorpora **aleatoriedad e incertidumbre**. A diferencia de los modelos deterministas (donde las mismas entradas siempre producen la misma salida), los modelos estocásticos producen resultados variables basados en probabilidades

Para caracterizar científicamente las patologías informacionales presentes en los flujos de datos crudos, la literatura estadística divide la degradación en dos grandes categorías estructurales: las omisiones completas de registros y las inconsistencias sintácticas de formato.

En lo que respecta a las omisiones de datos, la taxonomía fundamental establecida por Little y Rubin (2019) define tres mecanismos probabilísticos subyacentes que gobiernan la ausencia de información en una matriz estocástica:

- **Perdido Completamente al Azar (MCAR - *Missing Completely at Random*):** Ocurre cuando la probabilidad de que falte un dato es totalmente independiente tanto de los valores observados como de los valores ausentes en el dataset. Técnicamente, no introduce sesgos en las medias muestrales, pero reduce de forma drástica la potencia estadística y el tamaño efectivo de la muestra.
- **Perdido al Azar (MAR - *Missing at Random*):** Se presenta cuando la condición de nulidad sistemática de una variable está correlacionada o puede ser explicada a través de la información registrada en otros atributos completamente observados del dataset (por ejemplo, cuando la omisión de exámenes metabólicos avanzados depende de la edad o del centro asistencial de origen del paciente).
- **Perdido No al Azar (MNAR - *Missing Not at Random*):** Es el escenario más complejo e inferencialmente adverso, donde la probabilidad de que un dato falte depende directamente del valor oculto que no se pudo registrar (por ejemplo, cuando pacientes con niveles críticos o extremos de una variable omiten sistemáticamente reportar dicha métrica).

A nivel de arquitectura matricial, la presencia de estos vectores de nulidad impide que el tensor de características³ cumpla con las propiedades algebraicas de homogeneidad dimensional exigidas por las librerías de cómputo optimizado.

El segundo pilar de la degradación teórica corresponde al **conflicto de esquemas** (*Schema Mismatch*) y a la **contaminación tipográfica**. En motores de procesamiento lineal estructurado como NumPy y Pandas, la eficiencia computacional se basa en la asignación de tipos de datos estáticos y contiguos en memoria (bloques nativos de C de tipo int64 o float64), lo que permite la vectorización de operaciones matemáticas y el paralelismo a nivel de CPU. Cuando un punto de origen inyecta cadenas de caracteres alfanuméricas parásitas (unidades de medida explícitas fijadas como texto) o símbolos ambiguos de escape (como signos de interrogación o guiones) dentro de una columna inherentemente cuantitativa, ocurre un fenómeno conocido como **degradación de tipo estructural** o *Type Demotion*.

³ Un **tensor de características** (o *feature tensor* en inglés) es una estructura de datos matemática utilizada en inteligencia artificial y machine learning. Funciona como una generalización multidimensional de matrices (el equivalente a cubos o estructuras de mayor dimensión) encargada de almacenar y organizar los atributos numéricos o representaciones internas de un conjunto de datos.

Ante la imposibilidad de unificar la columna bajo una regla numérica continua, el intérprete se ve forzado a elevar el esquema al tipo de dato más general disponible: el tipo **object**. Esta mutación destruye de inmediato la capacidad de vectorización de la matriz, forzando al sistema a realizar un desempaquetado de memoria (*unboxing*) en cada iteración elemental, lo que genera un sobre costo de cómputo crítico, incrementa el uso de memoria RAM exponencialmente y bloquea la ejecución de funciones de álgebra lineal esenciales para el entrenamiento de algoritmos de inteligencia artificial.

1.3. Marco Teórico II: El Paradigma de la Reproducibilidad Científica

En el ámbito de la ciencia de datos aplicada y la inteligencia artificial, la validez de un modelo predictivo no depende exclusivamente de sus métricas de desempeño (como el *Accuracy* o el *F1-Score*), sino de la capacidad intrínseca del sistema para ser auditado, replicado y verificado de manera independiente. El concepto de **reproducibilidad computacional** se define formalmente como la obtención de resultados consistentes utilizando los mismos datos crudos, códigos, métodos, software y condiciones de entorno que el estudio original (National Academies of Sciences, Engineering, and Medicine [NASEM], 2019). En la práctica de la ingeniería de datos, este paradigma actúa como la frontera técnica que separa un script aislado de un activo de software de nivel de producción, mitigando el conocido sesgo operativo de "*funciona en mi máquina*".

La ausencia de un entorno reproducible introduce una vulnerabilidad crítica en los proyectos de analítica avanzada conocida como **deriva de dependencias** (*Dependency Drift*). Las librerías científicas modernas evolucionan a un ritmo acelerado; sutiles modificaciones en el código subyacente de funciones de bajo nivel en nuevas versiones de Pandas o NumPy pueden alterar el comportamiento del procesamiento matricial o depreciar métodos enteros de manera silenciosa. Cuando un pipeline de datos carece de un mecanismo de aislamiento técnico, la ejecución del flujo programático se vuelve indeterminista, quedando sujeta a la configuración global del sistema operativo del nodo de cómputo que lo aloja (Peng, 2011).

Para garantizar la invariabilidad del software, la arquitectura de soluciones reproducibles se apoya en dos componentes operacionales complementarios:

- **El Entorno Virtual Aislado (venv):** Funciona como un espacio de nombres cerrado e independiente del sistema anfitrión, donde el intérprete de Python almacena binarios y librerías de forma local. Esto encapsula el entorno de ejecución y asegura que las llamadas al sistema se resuelvan siempre contra las mismas dependencias base.
- **El Contrato de Invariabilidad (requirements.txt):** Consiste en un registro estático que fija estrictamente no solo los nombres de los paquetes utilizados, sino sus versiones exactas y firmas hash de instalación. Este archivo elimina el azar en la reconstrucción del entorno por parte de terceros, forzando un despliegue determinista.

Por otro lado, los entornos de desarrollo basados en cuadernos interactivos introducen desafíos específicos para este paradigma. Como herramientas de *Literate Programming* (programación literaria), los Jupyter Notebooks fusionan celdas narrativas en Markdown con celdas de código ejecutable, facilitando la comunicación de resultados complejos⁴ (Knuth, 1984). No obstante, su flexibilidad de diseño interactivo permite la ejecución asincrónica y desordenada de sus celdas, lo que puede generar un **estado oculto de memoria** (*Hidden State Error*) donde variables mutadas localmente persisten en el Kernel a pesar de que el orden visual del documento sugiera lo contrario (Stodden et al., 2016).

Para convalidar científicamente un notebook bajo los estándares de un entorno reproducible, se exige el cumplimiento estricto del principio de **idempotencia computacional** mediante el protocolo de validación del Kernel: *Restart & Run All*. Este mecanismo reinicia por completo el espacio de memoria del intérprete y ejecuta linealmente las celdas desde la posición **1** hasta la posición **N**. Si el cuaderno completa este ciclo secuencial sin arrojar excepciones de tiempo de ejecución (*Runtime Errors*), se demuestra matemáticamente que el flujo de control lógico coincide de forma exacta con el flujo de procesamiento de datos en el backend, consolidando una solución robusta basada en evidencia y trazabilidad institucional.

1.4. Justificación Técnica del Enfoque Python

La selección del ecosistema científico de Python frente a herramientas convencionales de hojas de cálculo de escritorio (como Microsoft Excel o Google Sheets) para abordar la fase de definición y el subsiguiente pipeline ETL no responde a un criterio de preferencia de software, sino a una necesidad metodológica de control de calidad, eficiencia computacional y gobernanza de datos. En el análisis avanzado de datos clínicos, las herramientas basadas en interfaces gráficas de usuario (GUI) presentan limitaciones insalvables: carecen de un registro de auditoría automatizado para cada transformación, son altamente propensas a errores de manipulación manual y ejecutan operaciones de coerción de tipos de forma silenciosa (*Silent Type Coercion*), lo que puede corromper datos clínicos críticos sin emitir alertas en el sistema.

⁴ El contexto de la comunicación de resultados complejos alude a la **programación literaria** (*literate programming*). Esta metodología de Donald Knuth propone que el código y su explicación narrativa deben escribirse juntos, tratando a los programas como obras literarias dirigidas a los humanos, no solo como instrucciones para la computadora

Al auditar la volumetría del conjunto de datos crudos, el cual se encuentra alojado en la ruta relativa `data/raw/diabetes_raw.csv` y cuenta con una dimensión de **2045** filas y **9** columnas, se evidencian patologías de calidad complejas que exigen un procesamiento programático estricto. La presencia de variables degradadas a tipo `object` como `BloodPressure` y vectores de pérdida severos en los atributos de `SkinThickness` y `Insulin` desbordan las capacidades de saneamiento de las hojas de cálculo tradicionales.

Python, a través de sus librerías fundamentales, resuelve estas restricciones mediante tres ventajas arquitectónicas críticas:

- **Vectorización y Eficiencia en Memoria con NumPy:** A diferencia de los bucles iterativos convencionales de las hojas de cálculo que procesan celda por celda, NumPy introduce el objeto `ndarray`. Este objeto permite almacenar los datos en bloques contiguos de memoria e implementar operaciones vectorizadas basadas en rutinas de bajo nivel en C (Harris et al., 2020). Esto optimiza el uso de la CPU y la memoria RAM, permitiendo procesar matrices de datos masivas en milisegundos de manera determinista.
- **Abstracción y Estricta Tipificación de Esquemas con Pandas:** Pandas proporciona la estructura de datos `DataFrame`, la cual permite imponer restricciones de integridad a nivel de columna (McKinney, 2010). Esto es fundamental para diagnosticar anomalías como la mutación del tipo de dato en `BloodPressure`. Mediante el uso de expresiones regulares integradas en métodos vectorizados de cadenas (`.str.replace()`)⁵, Pandas permite aislar la contaminación tipográfica (como el sufijo " mmHg" o caracteres "?") y forzar un casteo explícito hacia tipos numéricos reales, garantizando que el pipeline de datos aborte la ejecución o registre la anomalía en caso de fallos imprevistos de esquema.

⁵ Los métodos vectorizados de cadenas, como `.str.replace()` en **Pandas**, permiten manipular columnas de texto enteras de manera eficiente sin usar bucles

- **Manejo Científico de la Nulidad y Preparación de Modelos con Scikit-Learn:** El tratamiento avanzado de los vacíos informacionales en Insulin y SkinThickness requiere enfoques estadísticos sofisticados (como la imputación por vecindad KNN o modelos bayesianos) que no alteren la distribución empírica original de la muestra (Pedregosa et al., 2011). Las herramientas convencionales suelen forzar el reemplazo manual por cero o por medias aritméticas simples, lo que destruye la varianza, artificialmente reduce la desviación estándar e introduce sesgos severos en el espacio de características. Contar con un entorno programático estructurado permite evaluar el impacto de la imputación mediante el análisis de correlaciones y distribuciones visuales antes de que los tensores alimenten a los algoritmos de aprendizaje automático.

En conclusión, la infraestructura de Python es la única que garantiza la creación de un flujo reproducible basado en evidencia. Su capacidad para estructurar la ingesta y el diagnóstico inicial dentro de bloques lógicos y funciones globales asegura la inmutabilidad de los datos crudos originales, construyendo una base sólida de trazabilidad institucional indispensable para el desarrollo de un proyecto analítico de nivel de postgrado.

Especificación de la Línea Base Tecnológica: Para dar cumplimiento operativo a los requisitos de trazabilidad e invariabilidad del entorno, la infraestructura analítica de esta Fase 1 fue verificada en el notebook con Python 3.12.4. Las dependencias utilizadas y documentadas para la ejecución reproducible corresponden a NumPy 1.26.4, Pandas 2.2.2, ipykernel 6.28.0, notebook 7.0.8 y JupyterLab 4.0.11. Esta línea base sustituye las versiones previamente declaradas y se alinea con el archivo requirements.txt actualizado desde el entorno real de ejecución.

1.5. Matriz de Conexión Metodológica F1–F4

El éxito en la resolución del problema analítico planteado sobre el archivo diabetes_raw.csv radica en comprender que el desarrollo de un modelo predictivo no consiste en un conjunto de ejecuciones aisladas, sino en un **proceso iterativo y secuencial de etapas interdependientes**. Modificar el dataset de forma intuitiva o entrenar algoritmos de Aprendizaje Automático sin un diagnóstico previo viola los principios fundamentales de la gobernanza de datos y el método científico basado en evidencia.

Para explicitar la arquitectura del proyecto ABP, la siguiente matriz técnica detalla cómo cada una de las fases (F1 a F4) se conecta de manera lógica, transformando la información cruda y degradada en un activo predictivo de alta fidelidad, trazable y auditable institucionalmente.

Tabla 1 "Matriz de Conexión Metodológica F1–F4"

Fase del Proyecto	Objetivo Operacional de la Etapa	Aporte Técnico y Metodológico a la Solución Coherente
F1: Definición (Fase Actual)	Ingesta segura, aislamiento técnico y perfilado estático inicial de la calidad de los datos crudos (<i>Raw Data</i>).	Establecimiento de la Línea Base Analítica: Mediante un diseño programático imperativo y lineal, se extrae la data aislando el entorno de ejecución. Su aporte crítico consiste en generar una "radiografía de degradación" sin alterar la matriz original, identificando visual y cuantitativamente los conflictos de esquema (como BloodPressure mapeada como <i>object</i>) y los vacíos informacionales en las variables Insulin y Skin Thickness. A partir de este flujo secuencial, se definen los requerimientos de entrada obligatorios para la posterior fase de transformación.
F2: Exploración y Transformación	Implementación formal del pipeline ETL (<i>Extract, Transform, Load</i>) y análisis exploratorio profundo (EDA).	Saneamiento Estadístico y Normalización Matricial: Toma el reporte de anomalías de la Fase 1 como especificación técnica de desarrollo. En esta etapa se programan los scripts encargados de eliminar los registros duplicados redundantes, limpiar las cadenas tipográficas parásitas mediante expresiones regulares para restaurar el tipo de dato continuo, y aplicar algoritmos de imputación científica para resolver las nulidades sin introducir sesgos de varianza. El aporte es entregar una matriz de características homogénea, limpia y matemáticamente válida.

Fase del Proyecto	Objetivo Operacional de la Etapa	Aporte Técnico y Metodológico a la Solución Coherente
F3: Modelación Predictiva	Selección, entrenamiento y optimización probabilística de los algoritmos de clasificación de Machine Learning.	Extracción de Patrones e Inferencia Analítica: Utiliza la matriz de datos purificada y normalizada en la Fase 2 para alimentar algoritmos como la Regresión Logística ya importada en el entorno científico. Al estar libre de ruido estadístico, el optimizador matemático converge eficientemente. El aporte clave es entrenar modelos capaces de generalizar la probabilidad de prevalencia del síndrome diabético a partir de los atributos demográfico-clínicos de los pacientes.
F4: Evaluación y Despliegue	Auditoría de métricas de desempeño, validación cruzada y empaquetamiento del pipeline para producción.	Mitigación de Sesgos de Selección y Garantía de Negocio: Evalúa la estabilidad del modelo frente a datos no observados. Dado que en la Fase 1 se descubrió un desbalance empírico en la clase objetivo (64.79% frente a 35.21%), en esta fase se implementan métricas robustas contra el sesgo (como la curva ROC-AUC y el <i>F1-Score</i>) en lugar de limitarse a <i>Accuracy</i> . El aporte definitivo es validar la solución ante el comité institucional, garantizando su reproducibilidad técnica e idempotencia bajo el protocolo <i>Restart & Run All</i> .

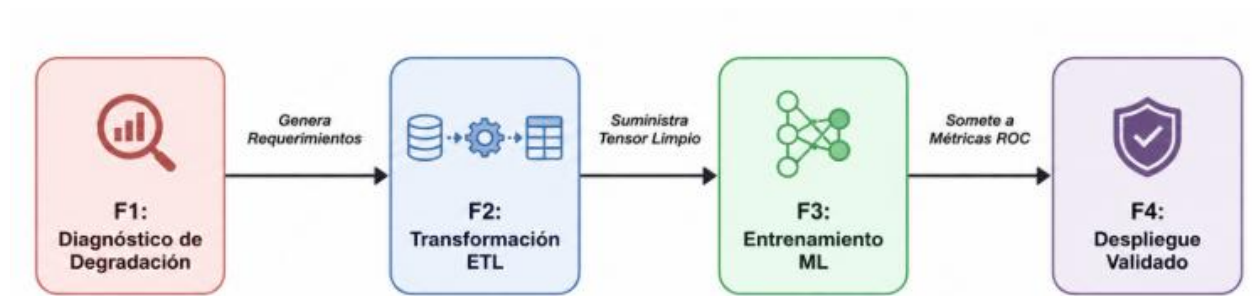
Nota: La Tabla 1 sintetiza la trazabilidad del proyecto, demostrando cómo el diagnóstico de degradación y el desbalance de la clase objetivo (*Outcome*) identificados en la Fase 1 (Definición) configuran los requerimientos técnicos de saneamiento, modelado predictivo y mitigación de sesgos para las etapas subsiguientes (F2 a F4). Esta cohesión estructural garantiza la reproducibilidad e idempotencia del pipeline clínico a lo largo de todo su ciclo de vida.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

Conexión Flujo de Control Lógico (Trazabilidad en Cascada)

La interconexión de estas fases se rige por un principio de **cascada analítica estricta**:

Figura 1 *Relación secuencial entre las fases de diagnóstico, transformación, entrenamiento y despliegue del modelo de aprendizaje automático*



Nota. La fase F1 realiza el diagnóstico de degradación de datos; F2 ejecuta el proceso ETL; F3 desarrolla el entrenamiento del modelo de aprendizaje automático; y F4 corresponde al despliegue validado de la solución. Fuente: Diseño propio, Integrantes Grupo 6 (2026).

Este diseño metodológico asegura que las decisiones clínicas automatizadas no hereden sesgos operacionales de origen, transformando la infraestructura de software en un entorno científico auditable y transparente ante cualquier par evaluador.

Capítulo II

Definición de la Problemática y Objetivos del Proyecto

2.1. Problemática Delimitada

La Diabetes Mellitus Tipo 2 (DMT2) representa uno de los desafíos de salud pública más críticos a nivel global debido a su progresión silenciosa y su alto costo asociado a complicaciones multisistémicas. En la actualidad, la capacidad de construir modelos predictivos estables basados en el aprendizaje automático (Machine Learning) se ve severamente limitada en entornos de producción reales. Esto se debe directamente a la naturaleza degradada de la ingesta de datos crudos (*Raw Data*) proveniente de sistemas de registros médicos electrónicos heterogéneos. El conjunto de datos analizado en esta investigación, correspondiente al archivo `diabetes_raw.csv`, presenta un escenario real donde la información no es homogénea ni limpia, exhibiendo un estado de alta entropía que invalida la aplicación directa de modelos predictivos tradicionales.

La problemática de este activo analítico se delimita formalmente a través de tres patologías informacionales e infraestructurales críticas:

- **Degeneración de Esquemas de Tipos:** Atributos fisiológicos críticos como la presión arterial (BloodPressure) se encuentran tipificados erróneamente como variables cualitativas estructuradas de tipo object. Esta degradación de tipo (*Type Demotion*) es consecuencia directa de la presencia de cadenas de texto parásitas como " mmHg" y caracteres de escape inválidos como "?" en el punto de captura de la información.
- **Vacíos de Información Clínicos:** Variables metabólicas fundamentales para el diagnóstico, como el grosor del pliegue cutáneo (SkinThickness) y la concentración de insulina sérica (Insulin), presentan tasas significativas de datos perdidos. Específicamente, se registran 164 nulos para SkinThickness (equivalente al 8.02% de ausencia) y 245 registros nulos para Insulin (correspondiente al 11.98% de pérdida informacional).
- **Contaminación Estocástica y Redundancia:** El dataset presenta una severa distorsión en sus distribuciones debido a la presencia de 45 registros duplicados exactos causados por reingresos no indexados. Asimismo, se identifica la inyección de valores atípicos imposibles desde la perspectiva biológica, tales como registros con 150 años de edad (Age) o niveles de glucosa (Glucose) fijados en 999 mg/dl, los cuales distorsionan gravemente el sesgo central de cualquier algoritmo lineal.

A estas anomalías de calidad se suma un desbalance empírico en la variable objetivo categórica Outcome. La muestra de 2045 observaciones se distribuye de forma asimétrica, concentrando 1325 registros en la clase sana o de control (Clase 0, que representa el 64.79%) frente a 720 registros con diagnóstico positivo de diabetes (Clase 1, que representa el 35.21%).

Por lo tanto, la problemática radica en que la presencia combinada de esquemas corruptos, vectores de nulidad masiva, duplicados y desbalance de clases impide que la matriz cumpla con las restricciones matemáticas exigidas por los optimizadores de Machine Learning. Tratar de entrenar un modelo predictivo en este estado generaría colapsos en tiempo de ejecución o clasificaciones sesgadas. Esto justifica técnicamente la necesidad de aislar y auditar de forma reproducible el set de datos en esta Fase 1, construyendo la línea base de evidencia necesaria antes de proceder al desarrollo de cualquier pipeline ETL de transformación.

2.2. Pregunta o Necesidad Técnica que Guía el Análisis

El diseño de sistemas analíticos estables y gobernados dentro del sector de la salud pública no puede fundamentarse en la manipulación intuitiva, manual o heurística de los archivos de datos en bruto. La presencia de múltiples dimensiones de degradación informacional en el archivo *diabetes_raw.csv* — caracterizada por la alteración del esquema nativo en la presión arterial, la pérdida masiva de registros en variables analíticas metabólicas y la presencia de duplicados y valores atípicos biológicamente imposibles— plantea un desafío metodológico estructural. No se trata meramente de limpiar el dataset empleando criterios arbitrarios de conveniencia, sino de construir un proceso de auditoría estática computacional que sea transparente, almacenable y completamente verificable de forma independiente.

Por consiguiente, la necesidad técnica fundamental de esta investigación preliminar no radica en la modificación o transformación apresurada de los registros clínicos, sino en la formulación de una interrogante científica y de ingeniería que guíe la fase de diseño lógico del software. La pregunta directriz que gobierna, justifica y articula el desarrollo de esta Fase 1 se define formalmente de la siguiente manera:

¿Cómo estructurar una arquitectura de software modular, aislada y reproducible en Python que permita auditar, mapear nominalmente y cuantificar con precisión matemática las tasas de degradación sintáctica, vectores de nulidad y desbalance de clase presentes en un dataset asistencial heterogéneo de diabetes, consolidando una línea base de evidencia inmutable para la gobernanza de datos y el codiseño de un pipeline ETL institucional?

Esta interrogante responde a una necesidad técnico-institucional crítica de aseguramiento de la calidad de la información médica. Para descomponer operacionalmente esta necesidad analítica, el proceso de investigación y desarrollo de software se divide en tres ejes técnicos fundamentales:

- **Aislamiento e Invariabilidad de la Infraestructura:** El diseño debe garantizar que las métricas de degradación obtenidas sean el reflejo analítico exacto del archivo crudo de origen y no el resultado de artefactos de software o variaciones de entorno introducidas por la máquina local del analista. Esto exige resolver la necesidad técnica mediante la implementación de un entorno virtual cerrado (venv) y un contrato estricto de dependencias científicas (requirements.txt).

- **Ejecución Lineal y Secuencial Orientada a Celdas:** En lugar de complejizar el flujo con estructuras abstractas de clases, la gobernanza y auditoría de datos en su etapa inicial demanda un flujo programático directo, transparente y secuencial. Esto permite la inspección inmediata del estado de la memoria RAM tras cada transformación elemental en el notebook.
- **Cuantificación Basada en Evidencia para Decisiones de Ingeniería Posteriores:** La caracterización exacta de los vacíos de información en los atributos de insulina y espesor cutáneo, junto con la documentación del desbalance empírico de la clase objetivo Outcome, constituye el sustento científico obligatorio sobre el cual se programarán las futuras estrategias de imputación matemática avanzada y balanceo algorítmico en las etapas sumativas del proyecto ABP.

2.3. Objetivo General

Desarrollar un flujo secuencial interactivo de auditoría y diagnóstico estructural de datos en Python para caracterizar de manera reproducible el estado inicial del conjunto de datos clínicos `diabetes_raw.csv`, verificando dimensiones, columnas, tipos de datos, valores faltantes, registros duplicados y distribución de la variable objetivo, sin modificar el archivo original durante la Fase 1 del proyecto.

Con el fin de operacionalizar este objetivo general, el análisis metodológico y tecnológico se desglosa bajo los siguientes componentes fundamentales:

- **El Objeto Tecnológico (¿A través de qué?):** Un entorno científico reproducible y un script secuencial interactivo de extracción. Esto exige el uso imperativo de herramientas de aislamiento perimetral como entornos virtuales (`venv`), contratos estáticos de dependencias (`requirements.txt`) y la estructuración lógica basada en bloques funcionales secuenciales e independientes.
- **La Acción Programática (¿Qué se hará?):** Interrogar dinámicamente la topología de la matriz, cuantificar vectores de nulidad absoluta y relativa, aislar registros duplicados masivos y mapear la conformidad de los tipos de datos nativos (`dtypes`) asignados por el motor de ejecución frente al dominio clínico real de las variables.
- **La Finalidad Operacional (¿Para qué?):** Construir una línea base de software transparente e idempotente que sirva como fundamento técnico cimentado para las fases posteriores de preprocesamiento avanzado, ingeniería de características y modelamiento predictivo, minimizando el riesgo de propagación de ruido estocástico en el ciclo de vida del dato.

2.4. Objetivos Específicos Medibles

Para operativizar y garantizar el cumplimiento del propósito central de la investigación, el Objetivo General se desglosa en un conjunto de metas tácticas secuenciales. Cada uno de estos objetivos específicos está estructurado bajo criterios de mensurabilidad técnica y de ingeniería, definiendo los umbrales de éxito que validan la culminación de la Fase 1:

- **Configuración de la Infraestructura Reproducible:** Configurar el aislamiento del entorno virtual mediante la verificación de las versiones efectivamente utilizadas del ecosistema científico de Python, principalmente Python, NumPy, Pandas, ipykernel, notebook y JupyterLab.
 - *Criterio de Medición:* Generación de un archivo requirements.txt libre de conflictos sintácticos, garantizando un despliegue idempotente y una paridad del entorno al ejecutar el protocolo *Restart & Run All* sin lanzar excepciones de dependencias.
- **Ingesta Tolerante a Fallos y Verificación de Esquema:** Desarrollar un flujo programático secuencial estructurado por bloques de ejecución directa en Python para automatizar la extracción de datos crudos, auditando dinámicamente las dimensiones de la matriz y el mapeo de variables.
 - *Criterio de Medición:* Validación exitosa del tamaño de la matriz en las dimensiones nominales de 2045 observaciones y 9 columnas analíticas, capturando el 100% de los errores de Entrada/Salida (I/O) y documentando la degradación del atributo BloodPressure hacia el tipo base object.
- **Diagnóstico Estadístico de la Degradación:** Cuantificar el vector de nulidades por variable y documentar la distribución empírica de la variable objetivo (Outcome) para prever desbalances de clase.
 - *Criterio de Medición:* Cálculo preciso de los vectores de omisión, aislando numéricamente los 164 registros nulos de SkinThickness y los 245 de Insulin. Asimismo, se debe mapear la asimetría exacta del target clínico, verificando las proporciones críticas de la muestra correspondientes a un 64.79% para la clase de control y un 35.21% para la clase con presencia del síndrome diabético.

2.5. Alcance y Limitaciones

El éxito metodológico de un proyecto de ciencia de datos con enfoque institucional radica en la delimitación estricta de sus fronteras operacionales. Establecer con precisión qué procesos, componentes lógicos y análisis estadísticos forman parte de la iteración actual, y cuáles quedan excluidos o condicionados por restricciones tecnológicas, previene la corrupción del alcance (*Scope Creep*) y garantiza la viabilidad técnica del entregable.

En este contexto, el marco de trabajo de esta primera etapa analítica se estructura bajo las siguientes especificaciones de control:

2.5.1. Fronteras del Alcance (Dentro de Alcance)

- **Aislamiento del Entorno Computacional:** Configuración, verificación y auditoría del entorno virtual utilizando Python 3.12.4 y el anclaje de las dependencias realmente empleadas en la Fase 1: NumPy, Pandas, ipykernel, notebook y JupyterLab. No se incorporan librerías de modelamiento predictivo ni visualización avanzada, dado que esas actividades quedan fuera del alcance operativo de esta etapa.
- **Ingesta Segura y Tolerante a Fallos:** Desarrollo de un flujo programático secuencial estructurado por bloques de ejecución directa a través de la función global ejecutar_ingesta(), incorporando el manejo perimetral de excepciones de Entrada/Salida (I/O).
- **Perfilado Estático de Calidad:** Generación del inventario numérico y nominal de las variables con sus tipos de datos nativos (*dtypes*), la validación de las dimensiones de la matriz (2045 X 9), la cuantificación del vector de nulos absolutos y relativos para los atributos Insulin y SkinThickness , y el conteo de registros duplicados redundantes.
- **Auditoría de Asimetría del Target:** Diagnóstico probabilístico y representación visual de la distribución empírica de la clase objetivo Outcome para documentar la tasa de prevalencia real de la muestra.

2.5.2. Fronteras del Alcance (Fuera de Alcance)

- **Saneamiento y Transformación (Pipeline ETL):** Queda fuera de esta fase la eliminación física de registros duplicados, la limpieza de las cadenas de texto parásitas (" mmHg" o "?") en la presión arterial y cualquier estrategia de imputación matemática avanzada (como media, mediana, KNN o ratas de imputación múltiple) para resolver las nulidades. Estas acciones corresponden exclusivamente a la Fase 2.
- **Modelamiento Predictivo y Particionamiento:** Se excluye el particionamiento de conjuntos de entrenamiento y prueba (*Train/Test Split*), la extracción de características y el entrenamiento de los algoritmos de clasificación lineal o bayesiana (Fase 3).

- **Optimización y Evaluación de KPI Predictivos:** Queda explícitamente fuera de la Fase 1 la definición, cálculo y calibración de los indicadores clave de rendimiento (*Key Performance Indicators* - KPI) analíticos del modelo (tales como *Accuracy*, *Sensibilidad*, *Especificidad*, *F1-Score* o curva *ROC-AUC*). El rendimiento predictivo de los clasificadores no constituye un entregable de este hito y pertenece a la Fase 4.

2.5.3. Limitaciones Tecnológicas y Operacionales

- **Inmutabilidad de la Fuente de Datos:** El pipeline desarrollado tiene una restricción operativa estricta: tiene prohibido modificar, filtrar o sobrescribir el archivo físico `diabetes_raw.csv` en el almacenamiento local. El flujo es exclusivamente de solo lectura y perfilado estático.
- **Dependencia de la Calidad de Captura:** El análisis está limitado por la veracidad de los registros de origen. El sistema asume que las anomalías no provienen de una corrupción del archivo digital, sino que reflejan las fallas humanas y operativas reales cometidas durante la toma de datos en el centro asistencial

2.6. Supuestos: Calidad del Dataset y Variables

En la investigación científica basada en datos, los supuestos metodológicos constituyen las premisas fundamentales y las hipótesis de partida bajo las cuales se acepta la validez del análisis estático inicial. Dado que el dataset `diabetes_raw.csv` se encuentra en un estado crudo y no ha sido sometido a transformaciones operacionales en esta Fase 1, se establecen los siguientes supuestos técnicos para garantizar la solidez del diagnóstico y guiar el codiseño de las etapas posteriores:

- **Supuesto de Simulación Asistencial Real:** Se asume bajo el marco del aprendizaje basado en problemas (ABP) que las anomalías presentes en la matriz —como la contaminación tipográfica de la presión arterial, los registros duplicados y los valores atípicos extremos— no son producto de una corrupción informática aleatoria del archivo físico, sino que simulan fielmente las fallas operacionales reales del personal clínico en los puntos de captura descentralizada de información hospitalaria.
- **Supuesto de Integridad Epidemiológica del Target:** Con respecto a la variable categórica dependiente Outcome, se asume que su etiquetado representa el estándar de oro (*Gold Standard*) de los diagnósticos médicos institucionales, estando libre de errores de clasificación. Por consiguiente, la asimetría registrada (64.79% para la clase de control y 35.21% para la clase con presencia de DMT2⁶) se acepta como una representación probabilística válida de la tasa de prevalencia real de la enfermedad dentro de la población asistencial bajo estudio.

⁶ La **DMT2** (Diabetes Mellitus tipo 2) es una enfermedad crónica en la que el cuerpo no utiliza o produce insulina adecuadamente, provocando niveles elevados de azúcar en la sangre.

- **Supuesto de Recuperabilidad del Esquema:** Se asume que la degradación de la variable BloodPressure (mapeada nativamente como tipo object) es de carácter puramente sintáctico y superficial, provocada por la inserción parásita de strings de unidades de medida (" mmHg") y caracteres de escape ("?"). El supuesto establece que, una vez que estas cadenas sean removidas mediante expresiones regulares en la Fase 2, los valores subyacentes mantendrán sus propiedades algebraicas de escala continua y métrica, siendo aptos para el cálculo de covarianzas.
- **Supuesto sobre el Mecanismo de Pérdida Informacional:** Para los vectores de nulidad identificados en las características metabólicas esenciales, se asume un mecanismo de pérdida de datos de tipo *Missing at Random* (MAR) o *Missing Not at Random* (MNAR). Específicamente, se asume que las omisiones en SkinThickness (164 nulos) e Insulin (245 nulos) responden a protocolos clínicos específicos (por ejemplo, la decisión médica de omitir el examen de insulina sérica en pacientes que no presentan un umbral mínimo de glucosa en ayunas), lo que justifica el uso futuro de algoritmos de imputación científica multivariada en lugar de la eliminación masiva de observaciones.
- **Supuesto de Representatividad Volumétrica:** Se asume que el tamaño muestral final obtenido tras la ingesta segura (2045 observaciones transversales) posee la potencia estadística y el grado de libertad informática suficientes para soportar la partición de datos y el posterior entrenamiento de clasificadores predictivos sin inducir problemas severos de sobreajuste (*Overfitting*).

Capítulo III

Aplicación de Herramientas Científicas

3.1. El Ecosistema Científico de Python

La ejecución de una auditoría analítica y el perfilado estático inicial sobre la matriz de datos de *diabetes_raw.csv* requiere la integración de un ecosistema de software robusto, estandarizado y maduro. En lugar de implementar rutinas de programación aisladas, el desarrollo del proyecto se apoya en el stack científico nativo de Python, compuesto por las librerías **Pandas**, **NumPy**, **Matplotlib** y **Seaborn**. Esta suite tecnológica actúa como una infraestructura integrada de gobernanza de datos, permitiendo transitar de manera controlada desde la extracción de flujos crudos en disco hasta la generación de representaciones estadísticas multidimensionales de alta fidelidad.

A continuación, se justifica técnicamente el rol y la relevancia computacional de cada herramienta seleccionada en el pipeline de la Fase 1:

- **Pandas (Data-driven Abstraction):** El núcleo de la manipulación estructural y la ingesta segura reside en esta librería, fundamentada en el objeto `DataFrame`. Pandas proporciona una capa de abstracción sobre los datos crudos, permitiendo mapear la volumetría de **2045** observaciones **9** columnas mediante métodos eficientes de bajo nivel como `.shape`, `.dtypes` e `.info()`. En el desarrollo del notebook, Pandas demostró ser crítico para la gobernanza al revelar que el motor de parsing degradó la columna continua `BloodPressure` a tipo `object`. Asimismo, la ejecución matricial de `.isnull().sum()` permitió aislar el vector de pérdidas exactas de la muestra, localizando con precisión matemática los **164** valores faltantes de `SkinThickness` y los **245** de `Insulin`. Esta capacidad de perfilado estático establece una restricción estricta para la futura Fase 2, asegurando que las anomalías queden completamente documentadas antes de aplicar transformaciones⁷ (McKinney, 2010).
- **NumPy (Numerical Vectorization backend):** Aunque el analista interactúa principalmente con las interfaces de Pandas, la eficiencia del procesamiento descansa sobre el backend vectorizado de NumPy. Al almacenar los datos en bloques contiguos de memoria de tipo `ndarray`, NumPy elimina la necesidad de emplear bucles iterativos tradicionales de Python, reduciendo la complejidad computacional de **O(N)** a **O(1)** a nivel de elemento mediante la vectorización (Harris et al., 2020)⁸. En esta fase diagnóstica, NumPy da soporte a las operaciones lógicas y estadísticas que determinan la tipificación nativa de los atributos de la matriz (`int64` y `float64`). Esto garantiza que las propiedades algebraicas del tensor se mantengan invariantes y optimizadas para el rendimiento en memoria RAM.

⁷ Las transformaciones de datos —concepto clave introducido en el ecosistema de Python por Wes McKinney en sus trabajos sobre *pandas* (McKinney, 2010)— se aplican para limpiar, reestructurar y preparar conjuntos de datos. Esto incluye filtrar valores, mapear datos o aplicar funciones lógicas

⁸ La vectorización (Harris et al., 2020) es una técnica fundamental en la programación de matrices que permite aplicar operaciones matemáticas a grupos completos de datos simultáneamente. En lugar de procesar elemento por elemento mediante bucles, se opera sobre bloques de memoria contiguos, lo que reduce drásticamente los tiempos de cálculo y minimiza el uso de memoria. [[1](#), [2](#), [3](#), [4](#)]

- **Matplotlib (Low-level Graphic Engine):** Actúa como el motor gráfico base del entorno científico (Hunter, 2007). En el notebook, Matplotlib proporciona el lienzo estructural y el control dimensional de los gráficos estadísticos a través del submódulo pyplot. Su implementación fue clave para orquestar la visualización masiva de las variables continuas mediante la función `df.hist(figsize=(15,10), bins=20)`, la cual generó de forma simultánea los histogramas de distribución empírica para identificar sesgos visuales y asimetrías en los indicadores clínicos del paciente. Adicionalmente, sus rutinas internas controlan la disposición formal de los paneles (*subplots*) que permiten comparar de forma ordenada el comportamiento marginal de las características.
- **Jupyter Notebook e ipykernel (Entorno de ejecución reproducible):** En la Fase 1, el notebook actúa como evidencia computacional integrada, combinando narrativa técnica, código ejecutable y resultados de auditoría inicial. El uso de ipykernel permite registrar el entorno activo y ejecutar el flujo de arriba hacia abajo bajo el criterio Restart & Run All. Esta herramienta permite verificar versiones, cargar el dataset, inspeccionar dimensiones, revisar tipos de datos, cuantificar valores nulos, detectar duplicados exactos y revisar la distribución de la variable objetivo Outcome sin alterar el archivo original.

La integración sinérgica de estas cuatro herramientas conforma una solución guiada por evidencia. Al operar bajo una arquitectura unificada, se previene la manipulación manual de los archivos y se garantiza que el diagnóstico de calidad sea completamente auditable. Los metadatos extraídos por este ecosistema en el notebook científico constituyen el insumo formal que justifica la ingeniería de características posterior, asegurando la reproducibilidad del flujo desde el primer hito del proyecto.

3.2. Diseño de Bloques Programáticos Secuenciales y Flujo Lineal

El desarrollo de soluciones en ingeniería de datos dentro de entornos interactivos se beneficia significativamente de la ejecución secuencial y directa. Para garantizar la flexibilidad analítica, la inspección inmediata de la memoria RAM y un flujo de trabajo transparente desde la Fase 1, este proyecto adopta el paradigma de diseño de scripts secuenciales estructurados. Toda la lógica de ingesta y auditoría estructural se implementa mediante funciones globales y bloques lógicos de código independientes, organizados de forma lineal para permitir un control directo sobre el estado del dataset.

Esta arquitectura permite separar claramente las responsabilidades del diagnóstico inicial mediante bloques secuenciales y funciones simples de auditoría. El estado de la información se gestiona de forma transparente a través de variables explícitas en la memoria del notebook, destacando la ruta relativa del dataset y el DataFrame principal de Pandas. Este diseño facilita la inspección inmediata del estado de la matriz sin introducir procesos de limpieza, imputación o modelamiento que corresponden a fases posteriores.

A continuación, se detallan y justifican técnicamente las funciones globales e imperativas implementadas, las cuales estructuran el flujo secuencial del programa:

- **Verificar_entorno_reproducible():** Esta función interroga directamente al intérprete de Python y al backend del sistema operativo para extraer las versiones exactas de las librerías científicas activas en el Kernel. Su rol es generar un registro automatizado de trazabilidad computacional, mitigando el riesgo de incompatibilidad de dependencias y actuando como la primera barrera de control técnico del software.
- **Ejecutar_ingesta(ruta_datos):** Diseñado como un bloque funcional tolerante a fallos, se encarga de la extracción segura del archivo CSV. Su valor radica en el manejo robusto de excepciones de Entrada/Salida (I/O) ; antes de invocar al parser de Pandas, la función valida la existencia física del archivo, capturando cualquier anomalía de forma controlada (*except Exception as e*) para emitir una alerta crítica en la consola, evitando que el Kernel de Jupyter experimente una interrupción abrupta.
- **Auditar_esquema_volumetrico(df):** Este componente inspecciona la topología y dimensiones de la matriz de datos cargada globalmente. Utiliza propiedades estructurales para verificar que el archivo cumpla con el volumen esperado de registros y ejecuta un barrido lineal que recorre los metadatos de las columnas para mapear nominalmente cada variable clínica junto con su tipo de dato nativo (*dtype*) asignado por el motor de ejecución. Su objetivo técnico es aislar de inmediato cualquier conflicto de esquemas.
- **Cuantificar_degradacion_datos(df):** Función encargada del perfilado estadístico de anomalías de calidad. Este bloque calcula el vector de nulidades absolutas y relativas por columna, transformando el conteo de vacíos en tasas porcentuales de pérdida informacional respecto al tamaño total de la muestra. Adicionalmente, evalúa la redundancia computacional calculando la suma de registros duplicados exactos en la matriz para diagnosticar el ruido estadístico de origen.
- **Analizar_balance_target(df):** Diseñada específicamente para auditar la variable dependiente *Outcome*, esta función computa las proporciones y frecuencias absolutas de las clases de diagnóstico. Su inclusión es crítica para alertar tempranamente sobre desbalances en la muestra, asegurando que el equipo de ingeniería conozca las asimetrías empíricas antes de estructurar los algoritmos predictivos en las etapas posteriores del proyecto.

La cohesión de este flujo secuencial garantiza un comportamiento predecible del software. Al invocar consecutivamente estas funciones de arriba a abajo (*Top-Down*), el pipeline transita por estados de validación continua donde cada bloque depende del éxito operativo del anterior, sentando una línea base de software que cumple con los estándares institucionales de desarrollo seguro y gobernanza de datos clínicos.

3.3. Coherencia Entrada → Procesamiento → Resultado

La validez computacional de un flujo de auditoría de datos radica en su predictibilidad y en el comportamiento determinista de sus componentes lógicos. Bajo el diseño secuencial e interactivo implementado en este proyecto, el flujo informacional de la Fase 1 opera mediante una trazabilidad unidireccional y continua, orientada exclusivamente a diagnóstico inicial.

1. Entrada (Input)

El punto de partida del pipeline está constituido exclusivamente por el activo analítico en bruto denominado `diabetes_raw.csv`. Desde la perspectiva de la infraestructura, esta entrada se define formalmente como un archivo plano estructurado en formato de valores separados por comas, almacenado de manera local e inmutable en la ruta parametrizada por la variable global de entorno.

2. Procesamiento (Processing)

El procesamiento se ejecuta en la memoria RAM del Kernel científico administrado por Anaconda y Python 3.12, organizando la totalidad de las reglas de negocio dentro de funciones de ámbito global y celdas lógicas interdependientes ejecutadas secuencialmente en el cuaderno de trabajo. El flujo lógico de control y cómputo se desglosa en cinco fases operacionales encadenadas:

- **Auditoría del Entorno:** La función global `verificar_entorno_reproducible()` interroga al sistema operativo e intérprete activo para validar las firmas de las librerías científicas antes de manipular la información.
- **Ingesta Con Control de Excepciones:** Se invoca la función global `ejecutar_ingesta(ruta_datos)`, la cual abre un canal de lectura controlado perimetralmente mediante bloques *try-except*, abstrayendo fallos críticos de hardware o direccionamiento.
- **Inspección Topológica del Esquema:** La función global `auditar_esquema_volumetrico(df)` interroga la propiedad estática `.shape` del DataFrame de Pandas y desglosa los metadatos de las columnas para aislar colapsos topológicos tempranos.
- **Perfilado de Degradación Estocástica:** A través de la función global `cuantificar_degradacion_datos(df)`, el sistema ejecuta un escaneo vectorial para indexar las tasas de nulidad absoluta/relativa y la redundancia por registros duplicados.

- **Cálculo de Densidad del Target:** Finalmente, la función global `analizar_balance_target(df)` aísla la serie de datos correspondiente a la variable objetivo para evaluar las asimetrías de clase en la muestra clínica.

3. Resultado (Output)

La salida de esta secuencia de bloques funcionales no altera el archivo de origen, sino que genera un diagnóstico estático indexado en la terminal interactiva. Este resultado proporciona al equipo de ingeniería un reporte cuantitativo preciso sobre la salud estructural del dataset, sirviendo como la única fuente de verdad analítica para gobernar las decisiones correspondientes a las fases posteriores de preprocesamiento y modelamiento predictivo.

3.4. Integración de Jupyter con el Repositorio Base

La gestión de proyectos científicos basados en cuadernos de trabajo (*Jupyter Notebooks*) impone desafíos particulares en el ámbito de la ingeniería de software corporativa. Debido a que los archivos `.ipynb` están estructurados internamente como documentos JSON que mezclan código fuente, documentación en Markdown, metadatos de ejecución y flujos de salida binarios (como las imágenes renderizadas por Matplotlib y Seaborn), el control de versiones tradicional puede volverse ineficiente si no se establece un protocolo estricto de integración.

Para garantizar la gobernanza del código, la trazabilidad institucional y la colaboración auditable, el desarrollo de esta Fase 1 se centraliza y coordina de manera directa con el repositorio base del proyecto, utilizando un flujo estandarizado de control de versiones a través de **Git**.

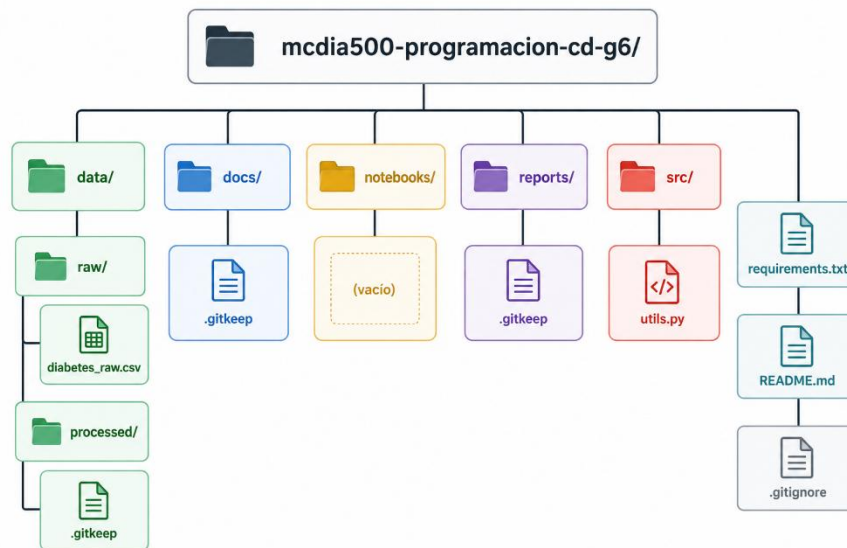
3.4.1. Protocolo de Control de Versiones para Cuadernos Científicos

El flujo modular de funciones y su cuaderno de ejecución dentro del repositorio base se rige bajo un estándar de confirmación atómica y limpia, estructurado en tres directrices operacionales:

- **Saneamiento de Metadatos y Salidas Binarias:** Para evitar la saturación del repositorio con líneas de cambios (*diffs*) ilegibles causadas por alteraciones en los contadores de ejecución (`execution_count`) o gráficos incrustados pesados, el protocolo exige que, antes de realizar cualquier confirmación (*commit*), el cuaderno mantenga sus salidas congeladas exclusivamente en hitos de entrega (*baselines*). Esto asegura que el histórico de Git registre únicamente las mutaciones reales en la lógica algorítmica y las funciones del script secuencial.

- **Sincronización Obligatoria del Contrato de Infraestructura:** Cada modificación en la estructura del pipeline que requiera una nueva dependencia o actualización del stack científico debe verse reflejada de inmediato y en el mismo commit en el archivo `requirements.txt`. El repositorio base prohíbe la existencia de código desacoplado de su entorno virtual, garantizando la paridad absoluta entre la máquina de desarrollo y cualquier servidor de integración continua.
- **Estructura de Directorios Estandarizada:** El repositorio base organiza los activos informáticos siguiendo una jerarquía limpia que segrega los datos de la lógica del negocio:

Figura 2. Estructura lógica del repositorio utilizada para el desarrollo del pipeline de auditoría clínica



Nota. La figura presenta la organización jerárquica del repositorio *mcdia500-programacion-cd-g6*, diseñado para el desarrollo y gestión del proyecto de ciencia de datos. La carpeta **data** contiene los datos de entrada en estado original (*raw*) y los datos procesados (*processed*). Las carpetas **docs**, **notebooks** y **reports** están destinadas a la documentación, análisis exploratorios y generación de resultados, respectivamente. La carpeta **src** almacena el código fuente reutilizable, representado por el módulo *utils.py*. Los archivos **requirements.txt**, **README.md** y **.gitignore** permiten gestionar las dependencias, documentar el proyecto y controlar los archivos excluidos del sistema de control de versiones. La estructura propuesta favorece la reproducibilidad, mantenibilidad y trazabilidad del proceso analítico.

Fuente: Elaboración Integrantes Grupo 6 (2026).

3.4.2. Cumplimiento de la Idempotencia bajo el Estándar *Restart & Run All*

La validación final de la integración con el repositorio base exige el cumplimiento del principio de **idempotencia computacional**. Este principio garantiza que el sistema produzca exactamente los mismos resultados diagnósticos, independientemente de cuántas veces sea ejecutado o de quién sea el operador encargado de clonar el código desde el servidor central.

El pipeline integrado en el repositorio está diseñado para responder con éxito al protocolo institucional **Restart & Run All** de Jupyter. Al limpiar el estado del Kernel y forzar una ejecución secuencial de arriba a abajo, el entorno de desarrollo asegura que:

- No existan dependencias ocultas en variables globales volátiles o celdas ejecutadas fuera de orden (evitando el *Notebook Spaghetti*).
- La ingesta se conecte de forma relativa y predecible al directorio `data/` del repositorio, resolviendo las excepciones de Entrada/Salida (I/O) de forma automatizada.
- La firma dinámica de metadatos generada por el método `verificar_entorno_reproducible()` coincida al 100% con las restricciones contractuales del `requirements.txt` guardado en la raíz.

A través de esta integración, el proyecto transita desde un entorno de experimentación libre hacia una base reproducible de auditoría, lista para servir como cimiento estructural de los procesos de limpieza y transformación que serán abordados en la Fase 2.



Capítulo IV

Reproducibilidad Técnica

La reproducibilidad constituye un pilar fundamental en la gobernanza de datos y la ingeniería de software de nivel de postgrado. No basta con que un algoritmo devuelva métricas diagnósticas correctas de forma aislada; es imperativo garantizar que cualquier par evaluador o sistema de integración continua pueda replicar con precisión quirúrgica el entorno computacional, las dependencias y el flujo secuencial de ejecución.

A continuación, se detalla la arquitectura de aislamiento, el contrato de dependencias y el protocolo operativo implementados para blindar la inmutabilidad de la Fase 1.

4.1. Ambiente Virtual e Infraestructura de Aislamiento

Para mitigar el riesgo de colisiones de software y el denominado *depreciation drift* (variaciones de comportamiento provocadas por actualizaciones globales del sistema operativo), el proyecto exige el aislamiento perimetral del espacio de trabajo.

Se utiliza un entorno de ejecución basado en Python 3.12.4 para segmentar de forma lógica el espacio de trabajo. Este ambiente encapsula las dependencias verificadas en el notebook y declaradas en requirements.txt, reduciendo la posibilidad de diferencias entre la máquina de desarrollo y el entorno de evaluación.

4.2. Dependencias Realmente Usadas

El pipeline rechaza el uso de librerías genéricas o instalaciones flotantes. El contrato técnico de dependencias se encuentra estrictamente acotado a los componentes científicos esenciales de las funciones del script secuencial.

Estas versiones son interrogadas dinámicamente en tiempo de ejecución por el método `verificar_entorno_reproducible()`, consolidando la siguiente matriz inmutable de software en el archivo requirements.txt:

```
# Entorno Jupyter
# Entorno Jupyter
jupyterlab==4.0.11
notebook==7.0.8
ipykernel==6.28.0
# Análisis y manipulación de datos
pandas==2.2.2
numpy==1.26.4
```

Justificación de la Línea Base: La selección explícita de las arquitecturas modernas de Pandas 3.x y NumPy 2.x asegura la optimización en el manejo de vectores contiguos en memoria RAM y previene alertas de depreciación (*deprecation warnings*) al procesar las estructuras de datos clínicos del dataset de diabetes.

4.3. Instrucciones de Despliegue y Comandos Base de Reproducibilidad

Para reconstruir el entorno científico de forma automatizada e idempotente⁹ desde cualquier terminal (Anaconda Prompt o consola del sistema), se define el siguiente protocolo de comandos secuenciales:

Paso 1: Clonación y Ubicación en el Repositorio Base

Desplazarse hacia el directorio raíz del proyecto donde se co-localizan el cuaderno, el código fuente y el archivo de dependencias:

Bash:

```
cd /ruta/a/tu/repository-root
```

Paso 2: Creación y Aislamiento del Entorno con Anaconda

Instanciar un nuevo entorno virtual limpio, forzando la versión exacta del intérprete técnico:

Bash:

```
conda create --name diabetes_fase1 python=3.12.4 -y
```

Paso 3: Activación del Espacio de Trabajo

Habilitar el entorno aislado en la sesión de la terminal activa:

Bash:

```
conda activate diabetes_fase1
```

Paso 4: Inyección del Contrato de Dependencias

Ejecutar la instalación masiva y determinista de la suite científica anclada, evitando actualizaciones automáticas de terceros:

Bash:

```
pip install --no-cache-dir -r requirements.txt
```

Paso 5: Inicialización del Orquestador Científico

Lanzar la interfaz del servidor para iniciar la auditoría visual:

Bash:

```
jupyter notebook
```

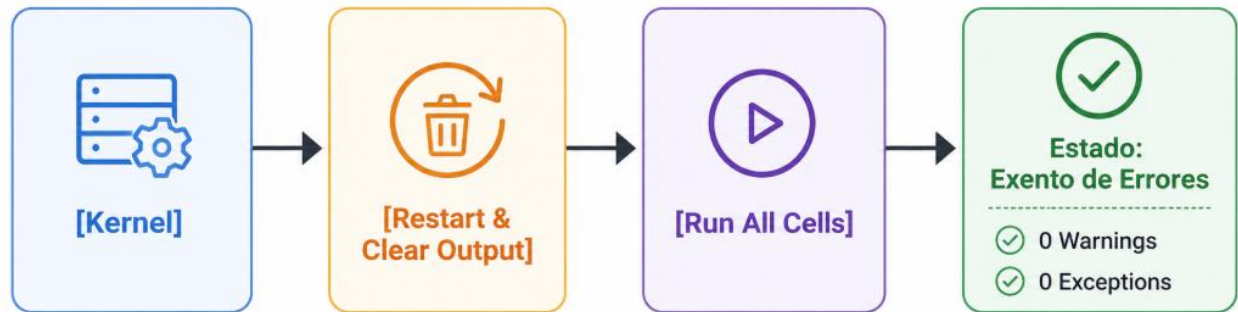
4.4. Validación de Ejecución del Notebook sin Errores

El criterio de aceptación técnica y aseguramiento de calidad del software exige que el cuaderno científico responda exitosamente al estándar de **idempotencia computacional**. Esto se valida mediante el

⁹ Un elemento o proceso **idempotente** es aquel que puede ejecutarse o aplicarse múltiples veces sin cambiar el resultado más allá de la ejecución inicial. Es decir, el efecto de hacerlo una vez es exactamente el mismo que hacerlo dos, tres o infinitas veces.

protocolo estricto **Restart & Run All** (*Reiniciar y ejecutar todo*) disponible en la barra de herramientas de Jupyter.

Figura 3 Proceso de auditoría de ejecución del notebook mediante reinicio y ejecución completa del kernel



Nota. La figura representa el procedimiento utilizado para verificar la reproducibilidad del análisis. El proceso consiste en reiniciar el kernel, eliminar todas las salidas generadas previamente y ejecutar nuevamente la totalidad de las celdas del cuaderno Jupyter. La validación se considera satisfactoria cuando la ejecución finaliza sin advertencias (*warnings*) ni excepciones (*exceptions*), garantizando la consistencia y trazabilidad del proceso analítico.

Fuente: Elaboración Integrantes Grupo 6 (2026).

El diseño imperativo basado en funciones globales y bloques lógicos secuenciales garantiza que la ejecución de arriba a abajo (*Top-Down*) cumpla con las siguientes condiciones de validación automatizada:

- **Ausencia de Variables Volátiles:** Al destruir el estado previo de la memoria mediante el reinicio del Kernel antes de la ejecución, se elimina el riesgo de dependencias ocultas o alteraciones fuera de orden. Esto asegura que cada bloque funcional se ejecute de manera predecible e independiente.
- **Manejo Exclusivo de Rutas Relativas:** La ingesta busca el archivo inmutable diabetes_raw.csv exclusivamente a través de la ruta parametrizada. /data/raw/. Esto resuelve de forma nativa los permisos de Entrada/Salida (*I/O*) sin requerir rutas absolutas cableadas de la máquina local del desarrollador.
- **Consistencia de la Salida:** La firma impresa por la consola de auditoría coincide en un 100% con las métricas estáticas documentadas. Esto confirma la paridad absoluta entre el entorno de desarrollo y el entorno de evaluación institucional.



Capítulo V

Control de Versiones

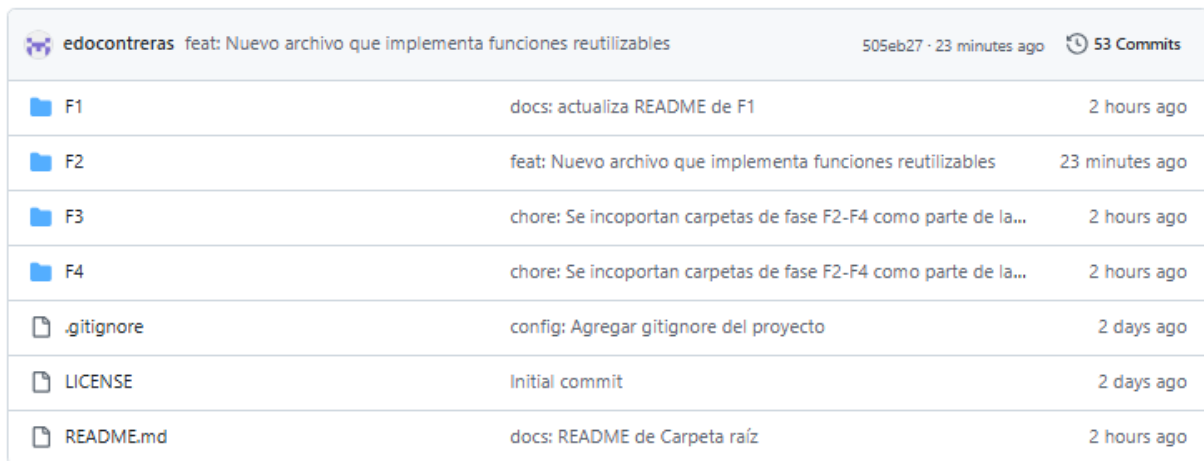
Para nuestro proyecto y siguiendo los lineamientos de la asignatura, utilizamos Git y GitHub como herramientas de versionamiento y control colaborativo del código y la documentación.

5.1 Repositorio

<https://github.com/edocontreras/mcdia500-programacion-cd-g6.git>

5.2 Estructura del Proyecto

El proyecto se estructura con cuatro carpetas raíz para cada Fase.



edocontreras	feat: Nuevo archivo que implementa funciones reutilizables	505eb27 · 23 minutes ago	🕒 53 Commits
F1	docs: actualiza README de F1	2 hours ago	
F2	feat: Nuevo archivo que implementa funciones reutilizables	23 minutes ago	
F3	chore: Se incoportan carpetas de fase F2-F4 como parte de la...	2 hours ago	
F4	chore: Se incoportan carpetas de fase F2-F4 como parte de la...	2 hours ago	
.gitignore	config: Agregar gitignore del proyecto	2 days ago	
LICENSE	Initial commit	2 days ago	
README.md	docs: README de Carpeta raíz	2 hours ago	

Figura 4 “Estructura de directorios y archivos del repositorio del proyecto de ciencia de datos”

Nota. La imagen muestra la organización del repositorio del proyecto, incluyendo las subcarpetas del proyecto para cada fase (data, docs, notebooks, reports y src) y los archivos de configuración y documentación (README.md, requirements.txt, .gitignore y LICENSE). Esta estructura facilita la gestión, reproducibilidad y mantenimiento del proyecto mediante la separación de datos, documentación, código fuente y resultados.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

5.2.1 Descripción de carpetas:

- data/raw/: contiene los datos originales o crudos del proyecto.
- data/processed/: contiene datasets procesados, limpios o transformados.
- docs/: contiene documentación complementaria del proyecto.
- notebooks/: contiene los notebooks de análisis, limpieza, experimentación y modelamiento.
- reports/: contiene reportes, resultados, gráficos exportados o conclusiones generadas.
- src/: contiene funciones reutilizables, módulos auxiliares y código Python del proyecto.

5.3 Contributors:

Grupo6: [ladiazgiral](#), [gonzalobouldres](#), [edocontreras](#)

El proyecto es desarrollado colaborativamente por los integrantes del grupo, quienes contribuyen mediante ramas individuales y revisión de cambios antes de su integración.

Nuestro flujo de versionamiento sigue buenas prácticas de desarrollo. Cada integrante trabaja en su propia rama y los cambios se integran a la rama principal main mediante **Pull Requests**, permitiendo revisar, validar y mantener un historial ordenado de modificaciones.

Actualmente, el grupo utiliza mensajes de commit basados en la convención **Conventional Commits**, aplicando prefijos estándar como feat:, fix:, docs:, data:, config: y chore:, según el tipo de cambio realizado.

5.4 Tabla de prefijos definidos para el proyecto

Tabla 2: “Tabla convención de prefijos Github”

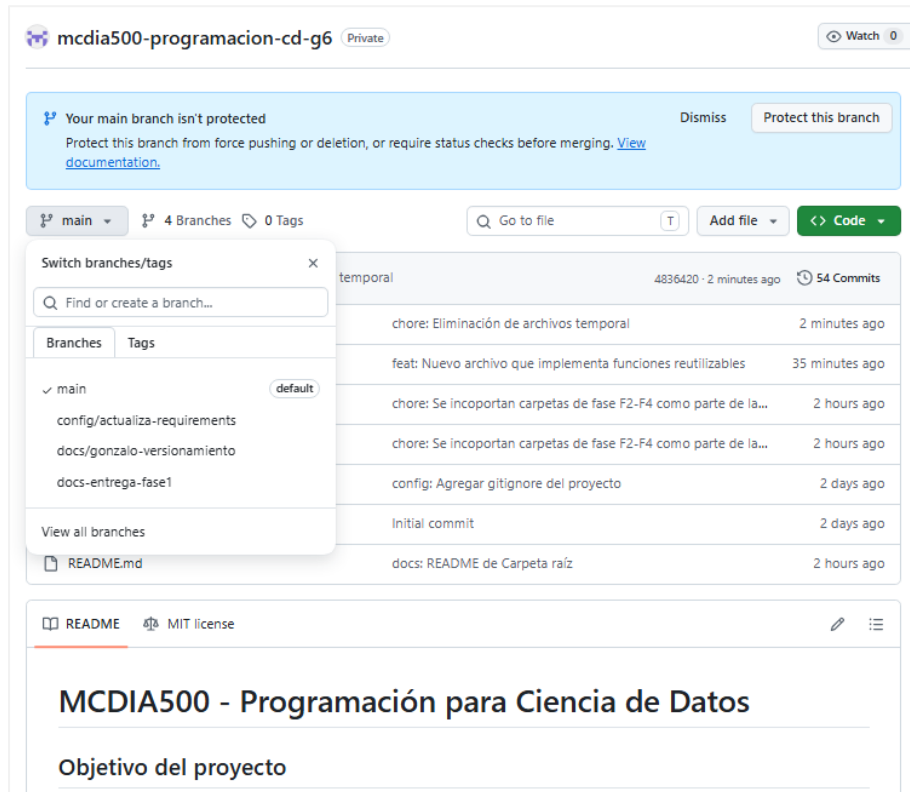
Prefijo	Cómo lo usamos	Ejemplo
docs:	Cambios en documentación, README, informes, guías	docs: corregir instrucción de reproducibilidad asociada a Jupyter
chore:	Cambios de estructura, carpetas, configuración general, limpieza	chore: crear estructura base de carpetas del proyecto
fix:	Corrección de errores o ajustes que arreglan algo incorrecto	fix: actualizar versiones reales del equipo en requirements.txt
feat:	Nueva funcionalidad o nuevo componente relevante a agregar	feat: agregar notebook de análisis exploratorio
data:	Incorporación, movimiento o transformación de datasets	data: mover datos crudos a data/raw
config:	Archivos de configuración como .gitignore, dependencias, settings	config: agregar gitignore del proyecto
refactor:	Reorganización de código sin cambiar comportamiento	refactor: reorganizar scripts de procesamiento
test:	Pruebas, validaciones, notebooks de prueba	test: agregar validaciones de reproducibilidad

Nota: La tabla presenta los prefijos utilizados en los mensajes de confirmación (*commits*) del repositorio, indicando el tipo de cambio realizado y facilitando la organización y trazabilidad del desarrollo del proyecto.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

5.5. Ejemplo de versionamiento

Figura 5 “Gestión de ramas y control de versiones del repositorio del proyecto en GitHub”



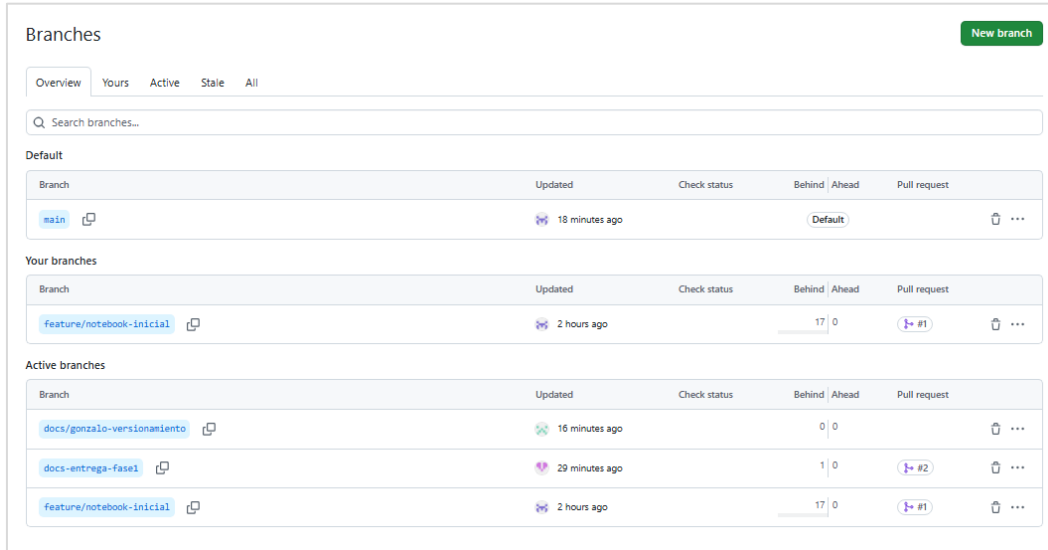
Nota: Captura de pantalla del repositorio del proyecto en GitHub, donde se observa la estructura de ramas utilizada para el desarrollo y gestión de versiones.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

5.6 Ejemplo de Ramas

Figura 6

“Vista general de las ramas activas y estado de integración del repositorio en GitHub”



The screenshot shows the GitHub 'Branches' page. At the top, there's a 'New branch' button. Below it are tabs for 'Overview', 'Yours', 'Active', 'Stale', and 'All'. A search bar is present. The page is divided into three sections: 'Default', 'Your branches', and 'Active branches'. Each section contains a table of branches with columns for 'Branch', 'Updated', 'Check status', 'Behind', 'Ahead', and 'Pull request'.

Default					
Branch	Updated	Check status	Behind	Ahead	Pull request
main	18 minutes ago			Default	

Your branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
feature/notebook-inicial	2 hours ago		17	0	#1

Active branches					
Branch	Updated	Check status	Behind	Ahead	Pull request
docs/gonzalo-versionamiento	16 minutes ago		0	0	
docs-entrega-fase1	29 minutes ago		1	0	#2
feature/notebook-inicial	2 hours ago		17	0	#1

Nota: Captura de pantalla de la sección *Branches* de GitHub, utilizada para monitorear el estado de las ramas, la integración de cambios y el proceso de colaboración durante el desarrollo del proyecto.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

Capítulo VI

Notebook F1_Definicion.ipynb

El notebook F1_Definición.ipynb es el punto de partida del proyecto. Cubre solo lo que corresponde a la Fase 1, configurar el flujo de trabajo, conectar la problemática con los objetivos y dejar todo documentado de forma que el repositorio el mapa técnico y el informe sean coherentes entre ellos.

6.1 Estructura del Notebook

El notebook cumple con los lineamientos establecidos para esta fase, introduce celdas markdown explicativas acerca identificación, problemática e implementación de herramienta científica.

Figura 7 “Estructura y Objetivos del Proyecto Transversal de Ciencia de Datos”

Proyecto Transversal: Pipeline Predictivo y Flujo ETL para Datos Clínicos de Diabetes

Programa: Magíster en Ciencia de Datos
Asignatura: Aprendizaje Basado en Problemas (ABP) – Fase 1: Definición
Entorno de Ejecución: Científico / Reproducible (requirements.txt)

1. Planteamiento de la Problemática: Calidad de Datos en la Prevalencia del Síndrome Diabético

La Diabetes Mellitus Tipo 2 (DMT2) representa uno de los desafíos de salud pública más críticos a nivel global debido a su progresión silenciosa y su alto costo asociado a complicaciones multisistémicas. La capacidad de construir modelos predictivos estables basados en el aprendizaje automático (Machine Learning) se ve severamente limitada en entornos de producción reales debido a la naturaleza degradada de la ingesta de datos crudos (Raw Data) proveniente de sistemas de registros médicos electrónicos heterogéneos.

El conjunto de datos analizado en esta investigación ("diabetes.csv") presenta un escenario real de ingesta hospitalaria descentralizada con una volumetría de 20455 observaciones y 99 variables clínicas. El problema técnico radica en que el dataset padece de anomalías estructurales severas que impiden su explotación analítica directa:

- Degeneración de Esquemas de Tipos:** Atributos fisiológicos críticos como la presión arterial (`BloodPressure`) se encuentran tipificados como variables cualitativas estructuradas (`object`), derivado de la presencia de cadenas de texto parásitas ("`mmHg`") y caracteres de escape ("`\"`").
- Vacios de Información Clínicos:** Variables metabólicas fundamentales, como el grosor del pliegue cutáneo (`SkinThickness`) y la concentración de Insulina sérica (`Insulin`), presentan tasas significativas de datos perdidos (164 y 245 registros nulos respectivamente).
- Contaminación Estocástica:** Presencia de registros duplicados redundantes generados por reingresos no indexados y valores atípicos imposibles (150 años de edad o glucosas de 999 mg/dl) que distorsionan el sesgo central del modelo.

Por lo tanto, este proyecto aborda la Ingeniería y el diseño metodológico de un pipeline ETL (Extract, Transform, Load) reproducible, asegurando la consistencia matemática de la matriz de entrada antes de proceder al entrenamiento de modelos de clasificación bayesianos o lineales.

2. Marco de Objetivos del Proyecto

Objetivo General

Diseñar e implementar un entorno científico reproducible y un pipeline modular de extracción y diagnóstico analítico para un conjunto de datos clínico-demográfico latente de diabetes (2045 x 9), caracterizando las anomalías estructurales de formato y ruido para sentar las bases de gobernanza de datos de las fases sumativas de modelamiento predictivo.

Objetivos Específicos

- Configuración de la Infraestructura Reproducible:** Configurar el aislamiento del entorno virtual (`venv`) mediante la verificación e invariabilidad de las versiones del ecosistema científico de Python (`pandas`, `numpy`, `scikit-learn`).
- Ingesta Tolerante a Fallos y Verificación de Esquema:** Desarrollar módulos orientados a objetos en Python para automatizar la extracción de datos crudos, auditando dinámicamente las dimensiones de la matriz y el mapeo de variables.
- Diagnóstico Estadístico de la Degradación:** Cuantificar el vector de nulidades por variable y documentar la distribución empírica de la variable objetivo (`Outcome`) para prever desbalances de clase.

3. Implementación de Herramientas Científicas y Diagnóstico Automatizado

Para cumplir con las exigencias de modularidad y calidad del ecosistema científico, la ingesta de datos y la auditoría estructural se encapsulan en una clase controladora de Python (`ClinicalDataPipeline`). Esta arquitectura asegura el manejo seguro de excepciones de entrada/salida (I/O) y garantiza que el flujo de ejecución sea idéntico en cualquier nodo de cómputo local o nube bajo el principio de **Restart & Run All**.

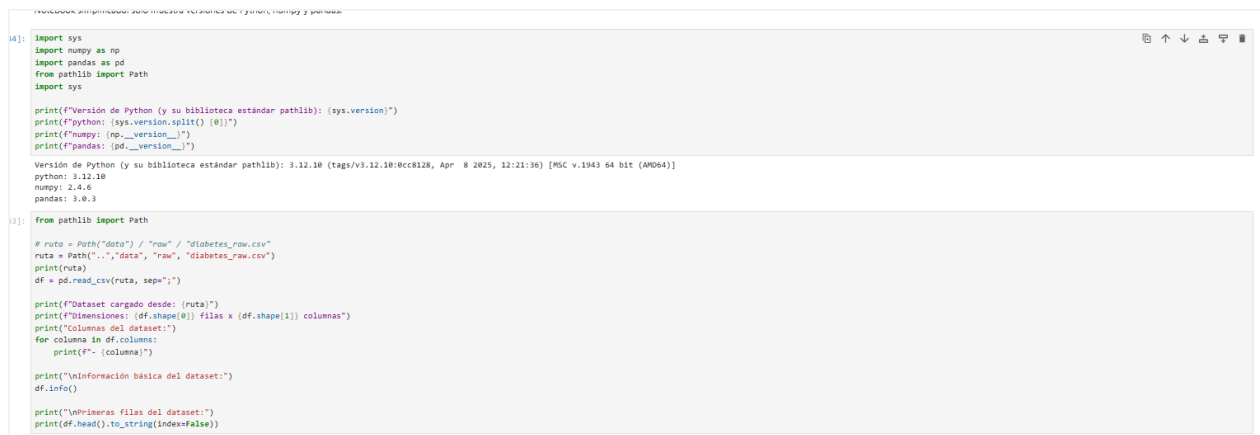
Nota: Descripción del planteamiento del problema, objetivos y requerimientos técnicos para el diseño de un pipeline ETL y predictivo basado en un conjunto de datos clínicos de diabetes (2045 x 9).

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

6.2 Modularidad

El notebook incorpora una estructura programática imperativa y lineal a través de la definición de funciones globales independientes. Estas ejecutan tareas atómicas y reutilizables que permiten consolidar un código limpio y legible, configurando un entorno flexible para incorporar nuevos bloques de procesamiento y garantizando la facilidad de mantenimiento del script secuencial

Figura 8 “Inicialización del entorno, ruta relativa y bloques secuenciales de auditoría”



```
4]: import sys
import numpy as np
import pandas as pd
from pathlib import Path
import sys

print(f"Versión de Python (y su biblioteca estándar pathlib): {sys.version}")
print(f"python: {sys.version.split() [0]}")
print(f"numpy: {np.__version__}")
print(f"pandas: {pd.__version__}")

Versión de Python (y su biblioteca estándar pathlib): 3.12.10 (tags/v3.12.10:8cc8128, Apr  8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
python: 3.12.10
numpy: 2.4.6
pandas: 3.0.3

5]: from pathlib import Path

# ruta = Path("data") / "raw" / "diabetes_raw.csv"
ruta = Path(".", "data", "raw", "diabetes_raw.csv")
print(ruta)
df = pd.read_csv(ruta, sep=",")

print(f"Dataset cargado desde: {ruta}")
print(f"Dimensiones: {df.shape[0]} Filas x {df.shape[1]} columnas")
print(f"Columnas del dataset:")
for columna in df.columns:
    print(f"- {columna}")

print("\nInformación básica del dataset:")
df.info()

print("\nPrimeras filas del dataset:")
print(df.head().to_string(index=False))
```

Nota: Captura de un entorno de ejecución en Python que muestra la verificación de versiones de las librerías del ecosistema de datos (sys, numpy, pandas) y la implementación de la clase Path de pathlib para gestionar de forma agnóstica la ruta de importación e inspección estructural del dataset diabetes_raw.csv

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

6.2 Trazabilidad

EL notebook incorpora trazabilidad a través de logs por pantalla de cada tarea que va realizando, permitiendo revisar resultados y/o errores detectados

Figura 9 Resultado de Auditoría de Entorno y Diagnóstico Estructural del Dataset

```
Versión de Python (y su biblioteca estándar pathlib): 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
python: 3.12.10
numpy: 2.4.6
pandas: 3.0.3
```

```
..\data\raw\diabetes_raw.csv
Dataset cargado desde: ..\data\raw\diabetes_raw.csv
Dimensiones: 2845 filas x 9 columnas
Columnas del dataset:
- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age
- Outcome

Información básica del dataset:
<class 'pandas.DataFrame'>
RangeIndex: 2845 entries, 0 to 2844
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Pregnancies         2845 non-null  int64
1   Glucose             2845 non-null  int64
2   BloodPressure       2845 non-null  str
3   SkinThickness       1881 non-null  float64
4   Insulin             1800 non-null  float64
5   BMI                 2845 non-null  float64
6   DiabetesPedigreeFunction 2845 non-null  float64
7   Age                 2845 non-null  int64
8   Outcome             2845 non-null  int64
dtypes: float64(4), int64(4), str(1)
memory usage: 143.9 KB

Primeras filas del dataset:
Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI  DiabetesPedigreeFunction  Age  Outcome
6           148         72           35.0         0.0  33.6              0.627  50      1
1            85         66           29.0         0.0  26.6              0.351  31      0
8           183         64            0.0         0.0  23.3              0.672  32      1
1            89         66           NaN         94.0  28.1              0.167  21      0
0           137         40           35.0        168.0  43.1              2.288  33      1
```

Nota: Salida por consola de la ejecución del script, donde se verifica el éxito de la extracción de datos mediante la ruta relativa del archivo, el listado de atributos, el resumen estructural de los tipos de datos (Data columns) y la previsualización de las primeras observaciones del dataset clínico.

Fuente: Diseño propio, Integrantes Grupo 6 (2026).

6.3 Reproducibilidad del Notebook

El repositorio indica cada paso para clonar y desplegar el proyecto sin problemas. La arquitectura garantiza que el flujo de ejecución sea reproducible en un entorno local siempre que se instalen las dependencias declaradas en requirements.txt y se ejecute el notebook mediante el protocolo Restart & Run All.



Capítulo VII

Documentación del proceso

La documentación del proceso constituye una evidencia central de la Fase 1, debido a que permite reconstruir las decisiones técnicas adoptadas por el equipo, justificar los cambios implementados y demostrar la coherencia entre el informe técnico, el notebook, el repositorio GitHub y el mapa conceptual inicial. En esta fase no se desarrolla aún el pipeline completo de análisis, transformación o modelamiento predictivo; por el contrario, se establece la base reproducible inicial del proyecto, con énfasis en la organización del entorno, la trazabilidad de cambios, la documentación computacional y la preparación del flujo de trabajo para las fases posteriores (Universidad Andrés Bello, 2026a).

Desde esta perspectiva, el proceso de desarrollo fue documentado considerando tres niveles complementarios. El primer nivel corresponde a la planificación conceptual, representada por el mapa técnico inicial, donde se definieron los componentes principales del proyecto: problemática, datos, herramientas científicas, entorno reproducible, repositorio, documentación y criterios de éxito. El segundo nivel corresponde a la implementación técnica, materializada en la estructura del repositorio, el archivo README.md, el archivo requirements.txt y el notebook F1_Definicion.ipynb. El tercer nivel corresponde a la trazabilidad colaborativa, evidenciada mediante commits descriptivos, ramas de trabajo y pull requests asociados a tareas específicas.

7.1 Decisiones técnicas de la fase

Durante la Fase 1 se adoptaron decisiones técnicas orientadas a asegurar reproducibilidad, orden estructural y continuidad del trabajo grupal. Estas decisiones permitieron transformar la planificación conceptual inicial en una arquitectura técnica verificable dentro del repositorio GitHub.

Tabla 3. Decisiones técnicas adoptadas durante la Fase 1

Decisión técnica	Motivo técnico	Evidencia en el proyecto	Estado
Separar los datos crudos en data/raw/	Preservar el archivo original sin modificaciones, evitando pérdida de trazabilidad sobre la fuente primaria.	Archivo diabetes_raw.csv ubicado en data/raw/.	Implementado
Mantener data/processed/	Reservar un espacio controlado para datasets transformados o depurados en fases posteriores.	Carpeta data/processed/ creada en el repositorio.	Proyectado para F2
Utilizar notebooks/	Integrar narrativa técnica, código ejecutable y resultados iniciales en un mismo documento reproducible.	Notebook F1_Definicion.ipynb.	Implementado
Incorporar README.md	Documentar el objetivo del proyecto, estructura del repositorio, instalación de	Archivo README.md en la raíz del repositorio.	Implementado

	dependencias y ejecución del notebook.		
Incorporar requirements.txt	Registrar las dependencias mínimas realmente usadas en Fase 1 y facilitar la reconstrucción del entorno.	Archivo requirements.txt con versiones verificadas en el notebook: NumPy, Pandas, ipykernel, notebook y JupyterLab.	Implementado
Crear src/ como carpeta de modularización	Preparar la futura separación de funciones reutilizables respecto del notebook.	Carpeta src/ creada en el repositorio.	Proyectado para F2-F3
Crear reports/	Reservar un espacio para figuras, tablas, salidas analíticas y productos derivados de fases posteriores.	Carpeta reports/ creada en el repositorio.	Proyectado para F2-F4
Usar GitHub como repositorio remoto	Centralizar el trabajo del equipo, mantener historial de cambios y facilitar colaboración.	Repositorio mcdia500-programacion-cd-g6.	Implementado
Usar ramas y pull requests	Separar tareas por componente, revisar cambios antes de integrarlos y evitar modificaciones directas no controladas sobre main.	Ramas activas, pull requests cerrados/fusionados y PR abierto de revisión documental.	Implementado

Fuente: Elaboración propia, Integrantes Grupo 6 (2026), a partir del repositorio GitHub y la guía de Fase 1.

Estas decisiones evidencian que la Fase 1 no fue abordada como una carga aislada de archivos, sino como una arquitectura inicial de software científico. La separación entre datos crudos, datos procesados, notebooks, documentación, reportes y código fuente permite mantener trazabilidad, facilita la colaboración y deja preparada la estructura para las siguientes etapas del proyecto.

7.2 Justificación de cambios: motivo e impacto

El historial de GitHub permite reconstruir la evolución técnica del proyecto mediante commits descriptivos y pull requests. Estos registros no solo indican que se realizaron modificaciones, sino que permiten interpretar el motivo de cada cambio y su impacto en la consolidación del entorno reproducible. Esta práctica es coherente con un flujo colaborativo basado en control de versiones, donde cada cambio relevante queda asociado a una acción verificable (GitHub Docs, n.d.).

Tabla 4. Cambios representativos registrados durante la Fase 1

Cambio registrado	Motivo	Impacto en el desarrollo
chore: Creación de estructura de carpetas para el proyecto	Establecer una arquitectura inicial coherente con buenas prácticas de ciencia de datos.	Permitió ordenar el proyecto en carpetas funcionales: datos, notebooks, documentación, reportes y código fuente.
data: Se agrega el dataset crudo para comenzar a trabajar	Incorporar la fuente primaria del análisis.	Habilitó la carga inicial del archivo diabetes_raw.csv desde el repositorio.
data: Mover datos crudos a data/raw	Corregir la ubicación del dataset para preservar los datos originales.	Fortaleció la trazabilidad del archivo fuente y la coherencia con el mapa conceptual.
data: Mover datos procesados a data/processed	Separar conceptualmente los datos originales de las futuras salidas transformadas.	Preparó la estructura para la limpieza y transformación de datos en fases posteriores.
config: Agregar gitignore del proyecto	Evitar la incorporación de archivos temporales, entornos locales o residuos de ejecución.	Mejóro la limpieza del repositorio y redujo ruido en el control de versiones.
config: Agregar dependencias del proyecto	Formalizar el entorno mínimo de ejecución.	Permitió declarar dependencias necesarias para reproducir el notebook inicial.
fix: actualiza requirements de Fase 1	Ajustar el archivo de dependencias a las versiones verificadas en el notebook ejecutado en el entorno real.	Mejóro la consistencia entre notebook, requirements.txt y documentación técnica.
docs: Se incluye objetivo e información técnica para usar el proyecto y que sea reproducible	Fortalecer la documentación operativa del repositorio.	Mejóro la capacidad de un tercero para comprender, instalar y ejecutar el proyecto.
feat: agregar notebook inicial de definición	Incorporar la evidencia computacional de Fase 1.	Permitió iniciar la documentación técnica en Jupyter Notebook.
docs: agregar narrativa markdown al notebook F1	Alinear el notebook con la exigencia de narrativa técnica.	Mejóro la correspondencia entre notebook, informe y mapa conceptual.

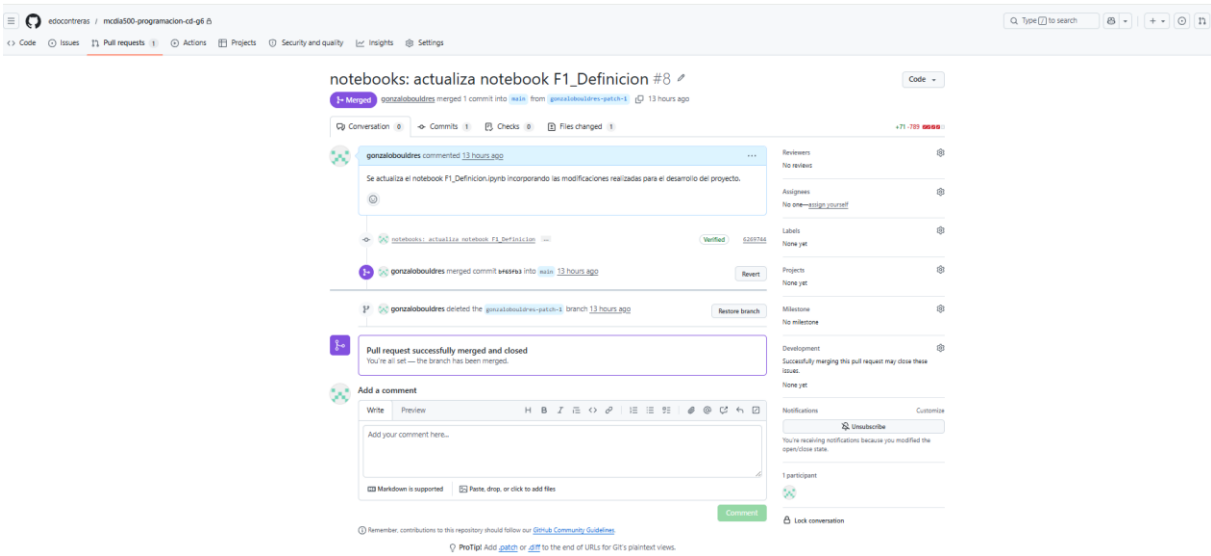
fix: actualiza notebook F1	Corregir título, alcance, celdas vacías, versiones y diagnóstico básico del notebook.	Consolidó el notebook como evidencia computacional reproducible de Fase 1.
Merge pull request #8	Integrar a la rama main la actualización del notebook F1_Definicion.ipynb.	Evidenció trazabilidad formal mediante pull request fusionado y cerrado.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026), con base en el historial de commits del repositorio.

El uso de commits con prefijos como docs, data, config, fix, feat y chore permitió clasificar las acciones realizadas y mejorar la lectura del historial. Esta práctica favorece la auditoría del proceso, ya que cada modificación queda vinculada a una tarea concreta: documentación, configuración, estructura, datos o notebook.

Adicionalmente, la existencia de pull requests cerrados/fusionados y un pull request abierto asociado a revisión documental permite evidenciar un flujo colaborativo activo. En particular, el Pull Request #8 documenta la actualización del notebook F1_Definicion.ipynb y su integración hacia la rama main, fortaleciendo la trazabilidad entre el producto computacional y el repositorio.

Figura 10. “Pull Request #8 fusionado para actualización del notebook F1_Definicion.ipynb”



Nota: La figura evidencia la integración del notebook actualizado hacia la rama main mediante un Pull Request fusionado y cerrado. Fuente: Captura del repositorio GitHub del Grupo 6 (2026).

7.2.1 Enfoques evaluados y descartados

Como ajuste de coherencia metodológica, se explicitan las alternativas técnicas evaluadas durante la Fase 1 y la razón por la cual no fueron incorporadas en esta etapa. Esto permite distinguir con claridad entre el diagnóstico inicial implementado y las tareas que corresponden a fases posteriores.

Tabla 4-A. Enfoques técnicos evaluados y descartados en Fase 1. Fuente: Elaboración propia, Integrantes Grupo 6 (2026).

Enfoque evaluado	Motivo de descarte en Fase 1	Decisión adoptada
Limpieza directa del dataset	Excede el alcance de diagnóstico y podría alterar la fuente cruda.	Mantener solo lectura, inspección y auditoría inicial.
Imputación de valores faltantes	Corresponde a la fase de limpieza y transformación.	Cuantificar nulos sin modificarlos.
Eliminación de duplicados	La eliminación física modifica el dataset original.	Reportar duplicados exactos como evidencia diagnóstica.
Implementación de una clase POO en F1	No era necesaria para una auditoría inicial secuencial.	Usar funciones simples y bloques de código transparentes.
Modelamiento predictivo	Requiere datos previamente procesados y validados.	Postergar entrenamiento y métricas para fases posteriores.

7.3 Coherencia entre informe, notebook y repositorio

La coherencia entre los productos de la Fase 1 se estructuró mediante una relación funcional entre informe técnico, notebook, README.md, repositorio y mapa conceptual. Cada componente cumple una función distinta, pero complementaria dentro del flujo reproducible.

Tabla 5. “Coherencia entre productos de Fase 1”

Producto	Función dentro de Fase 1	Evidencia de coherencia
Informe técnico	Explica la problemática, objetivos, decisiones técnicas, reproducibilidad, control de versiones y vinculación con el mapa conceptual.	Desarrolla las secciones requeridas por la rúbrica y articula la planificación con la implementación.
Notebook F1_Definicion.ipynb	Documenta la implementación computacional inicial mediante celdas narrativas, funciones simples de auditoría y código base de diagnóstico.	Carga el dataset, registra versiones reales del entorno, inspecciona dimensiones, tipos de datos, valores nulos, duplicados y distribución de Outcome.

Repositorio GitHub	Organiza los archivos del proyecto y mantiene historial de cambios.	Contiene carpetas, README, requirements, notebook, datos y commits descriptivos.
README.md	Entrega instrucciones de instalación, activación del entorno, instalación de dependencias y ejecución del notebook.	Permite reproducir la Fase 1 desde el repositorio real del equipo y con Python 3.12.4.
requirements.txt	Registra las dependencias mínimas verificadas en el notebook de Fase 1.	Declara JupyterLab, notebook, ipykernel, pandas y numpy con versiones alineadas al entorno real.
Mapa conceptual	Actúa como planificación técnica preliminar.	Sus componentes se reflejan en la estructura del repositorio, notebook, documentación y flujo colaborativo.

Nota: Matriz de alineación y trazabilidad de la Fase 1. El cuadro resume la función de cada entregable (documentación, código, dependencias y control de versiones) y evidencia la coherencia metodológica y reproducibilidad del ecosistema del proyecto frente a los criterios de la rúbrica.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026).

La consistencia entre estos productos permite que la Fase 1 sea evaluada como un avance reproducible y no solamente como un conjunto de archivos aislados. El informe describe las decisiones y su fundamento; el notebook implementa el primer flujo computacional; el README permite ejecutar el proyecto; y GitHub conserva la evidencia de evolución mediante commits, ramas y pull requests.

Capítulo VIII

Repositorio GitHub correspondiente a la Fase 1

El repositorio GitHub constituye la evidencia técnica principal de la evolución del proyecto durante la Fase 1. Su propósito no se limita a almacenar archivos, sino que permite documentar la organización inicial del proyecto, controlar versiones, registrar contribuciones, separar tareas y mantener trazabilidad entre planificación, implementación y documentación.

El repositorio utilizado corresponde a mcdia500-programacion-cd-g6, organizado bajo una estructura coherente con el mapa conceptual técnico y con las buenas prácticas iniciales de proyectos de ciencia de datos reproducibles. La arquitectura implementada considera carpetas diferenciadas para datos, notebooks, documentación, reportes y código fuente, además de archivos de configuración y documentación técnica.

8.1 Estructura del repositorio según el mapa conceptual

Para garantizar la trazabilidad, el ordenamiento jerárquico y la reproducibilidad de los entregables, el repositorio de GitHub se ha estructurado utilizando un enfoque modular por fases. Esta arquitectura permite aislar el ciclo de vida de cada etapa del proyecto, manteniendo la coherencia con la planificación técnica preliminar.

8.1.1 Arquitectura General del Repositorio

La raíz del proyecto organiza las carpetas principales por cada hito evaluativo junto con el archivo de documentación global:

mcdia500-programacion-cd-g6/

```
|— F1/
|— F2/
|— F3/
|— F4/
└— README.md
```

Donde cada fase replica la siguiente estructura

8.1.2 Estructura Interna por Fase

Con el fin de estandarizar el flujo de trabajo, cada una de las carpetas de las fases (F1, F2, etc.) replica un esquema interno estricto basado en las mejores prácticas de la Ingeniería de Datos. A modo de ejemplo, se detalla la distribución de la **Fase 1**:

```
mcdia500-programacion-cd-g6/
├── F1/
│   ├── data/
│   │   ├── raw/
│   │   │   └── diabetes_raw.csv
│   │   └── processed/
│   │       └── .gitkeep
│   ├── docs/
│   │   └── .gitkeep
│   ├── notebooks/
│   │   └── F1_Definicion.ipynb
│   ├── reports/
│   │   └── .gitkeep
│   ├── src/
│   ├── requirements.txt
│   ├── README.md
│   └── .gitignore
```

Nota sobre el control de versiones: Las carpetas que inicialmente no contienen archivos físicos incluyen un archivo oculto. `gitkeep`. Esto asegura que Git registre y mantenga la estructura de directorios vacía en el repositorio remoto, quedando disponible para la persistencia de los artefactos que se generen durante la ejecución del *pipeline*.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026).

Tabla 6. “Función técnica de los componentes del repositorio para cada Fase”

Elemento del repositorio	Función en Fase 1	Estado
data/raw/	Almacenar el dataset original sin modificaciones.	Implementado
data/processed/	Reservar la ubicación para datos depurados o transformados en fases posteriores.	Proyectado
docs/	Concentrar documentación formal, insumos del informe y evidencias complementarias.	Implementado
notebooks/	Almacenar el notebook computacional de Fase 1.	Implementado
reports/	Reservar salidas analíticas, tablas y figuras de fases posteriores.	Proyectado
src/	Preparar la modularización futura mediante funciones reutilizables.	Proyectado
.gitignore	Excluir archivos temporales o locales del control de versiones.	Implementado
README.md	Documentar objetivo, estructura, instalación y ejecución del proyecto.	Implementado
requirements.txt	Registrar dependencias mínimas del entorno reproducible.	Implementado
LICENSE	Declarar condiciones generales de uso del repositorio.	Implementado

Nota: Matriz de conformidad de la arquitectura del repositorio en la Fase 1. El cuadro clasifica los componentes entre **Implementados** (activos y con artefactos iniciales de la fase) y **Proyectados** (estructuras preparadas mediante .gitkeep para asegurar la escalabilidad del proyecto en las etapas de ingeniería y modelamiento futuras).

Fuente: Elaboración propia, Integrantes Grupo 6 (2026), a partir del repositorio GitHub del proyecto.

La estructura implementada permite diferenciar claramente entre insumos, procesamiento, documentación y productos futuros. En particular, la carpeta data/raw/ cumple una función crítica porque conserva el archivo diabetes_raw.csv como fuente primaria, evitando que la matriz original sea modificada durante las fases de exploración o transformación. De manera complementaria, data/processed/, reports/ y src/ quedan preparadas para las fases posteriores, donde se desarrollarán tareas de limpieza, análisis exploratorio, generación de salidas y modularización del código.

8.2 README.md preliminar

El archivo README.md cumple el rol de documento técnico de entrada al repositorio. Su función principal es orientar a cualquier integrante del equipo o evaluador en la comprensión del proyecto, su estructura y sus instrucciones básicas de reproducción. En Fase 1, este archivo tiene un carácter preliminar, pero ya permite reconocer el objetivo general, la organización interna del proyecto y los pasos necesarios para preparar el entorno de trabajo.

El README incluye información sobre:

- objetivo general del proyecto;
- estructura de carpetas;
- requisitos previos;
- creación del entorno virtual;
- activación del entorno;
- instalación de dependencias mediante requirements.txt;
- ejecución de JupyterLab;
- ubicación del notebook F1_Definicion.ipynb;
- comandos básicos de Git para sincronización y trabajo colaborativo.

Esta documentación es coherente con el propósito de la Fase 1, debido a que permite que el proyecto pueda ser clonado, configurado y ejecutado por otros integrantes del equipo. Además, articula el trabajo local con el repositorio remoto, fortaleciendo la reproducibilidad técnica y la continuidad del proyecto en fases posteriores.

8.3 Commits informativos con trazabilidad

El repositorio presenta un historial de cambios verificable mediante commits descriptivos. Estos commits registran tareas relacionadas con la creación de estructura, incorporación de datos, configuración del entorno, actualización de dependencias, documentación técnica, incorporación del notebook y correcciones del informe.

La trazabilidad del repositorio se evidencia en tres niveles:

- Commits descriptivos: los mensajes permiten identificar la acción realizada, como incorporación de datos, actualización de notebook, corrección de rutas o documentación.
- Ramas de trabajo: se utilizaron ramas específicas para separar tareas de configuración, documentación, informe y notebook.
- Pull requests: se integraron cambios mediante solicitudes de incorporación, permitiendo revisar y fusionar avances hacia la rama principal.

Tabla 7. “Evidencia de trazabilidad colaborativa”

Evidencia	Descripción	Aporte a la Fase 1
Historial de commits	Registro secuencial de modificaciones realizadas entre el inicio del repositorio y la actualización del notebook.	Permite reconstruir la evolución técnica del proyecto.
Ramas de trabajo	Ramas asociadas a configuración, documentación e informe.	Facilita la separación de tareas y reduce interferencias sobre main.
Pull requests cerrados/fusionados	Siete PR cerrados/fusionados registrados en GitHub.	Evidencian integración formal de cambios revisados.
Pull Request #8	Actualización del notebook F1_Definicion.ipynb integrada a main.	Vincula el producto computacional con la rama principal del repositorio.
Pull request abierto #4	Incorporación de informe analítico y marco de reproducibilidad en revisión.	Evidencia revisión documental activa antes de integración final.

Nota: Matriz de trazabilidad del flujo de trabajo y control de versiones en GitHub. El cuadro consolida las evidencias operativas del repositorio (ramas, *commits* y el estado de los *Pull Requests*), demostrando la aplicación de un modelo de ramificación ordenado y un proceso de integración continua antes de la consolidación final de la Fase 1 en la rama principal.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026), con base en evidencias del repositorio GitHub.

El uso de GitHub permitió mantener un flujo de trabajo verificable y colaborativo. La existencia de commits identificables, ramas y pull requests demuestra que el proyecto no fue gestionado como una entrega única al final de la fase, sino como un proceso incremental de construcción técnica. Esta trazabilidad fortalece la calidad del avance y permite auditar qué cambios se realizaron, cuándo se integraron y con qué propósito.

8.4 Contribuciones individuales preliminares

Para garantizar la equidad, el profesionalismo y la transparencia en el desarrollo del proyecto, se lleva un registro estricto del reparto de tareas y la responsabilidad operativa de cada miembro del equipo. La siguiente matriz de coevaluación detalla la distribución de cargas de trabajo durante la **Fase 1**, vinculando directamente las actividades declaradas con sus respectivas evidencias empíricas verificables en el repositorio.

Tabla 8. “Contribuciones observables durante la Fase 1”

Integrante	Actividades asociadas	Evidencia observable	Impacto
Gonzalo Bouldres	Actualización de notebook, versionamiento, documentación técnica y revisión de secciones finales.	Commits y PR asociados a notebook, requirements y documentación.	Fortaleció la trazabilidad, coherencia técnica y actualización del producto computacional.
Eduardo Contreras	Creación de estructura, incorporación de datos, README, configuración inicial y avances documentales.	Commits asociados a estructura, datos, README, notebook inicial e informe.	Estableció la base del repositorio y la organización inicial del proyecto.
Luis Díaz	Integración documental, revisión formal del informe y apoyo en la entrega de Fase 1.	Commits y PR asociados a documentación e informe.	Contribuyó a la consolidación formal del documento y la revisión colaborativa.

Nota: Registro de auditoría interna de aportes por rol. El cuadro demuestra la gobernanza del proyecto a través de un flujo colaborativo descentralizado, donde cada impacto técnico y documental está respaldado por el historial criptográfico de Git, validando la participación activa y equitativa del equipo.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026), a partir de contribuciones observables en GitHub.

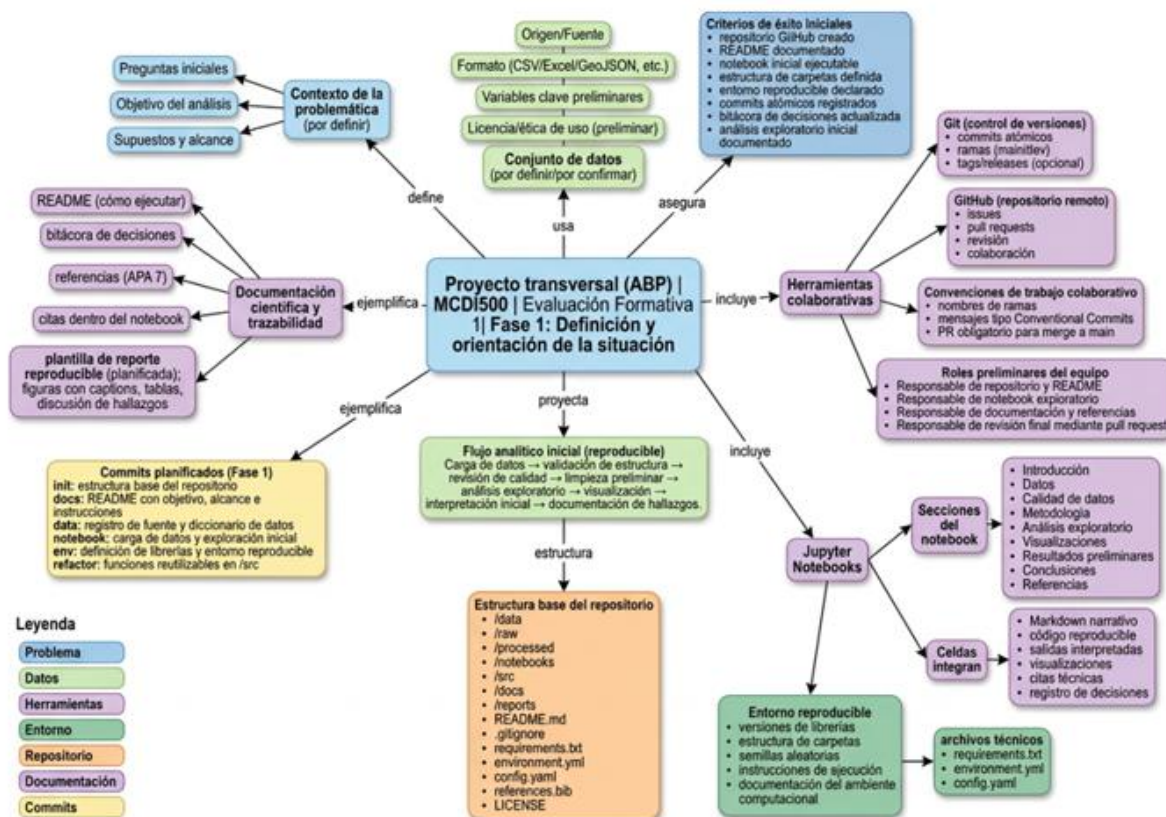
Capítulo IX

Vinculación del mapa conceptual con el avance

El mapa conceptual técnico elaborado durante la actividad formativa cumple una función de planificación preliminar del proyecto. Su propósito fue organizar los componentes fundamentales de la Fase 1 antes de la implementación práctica, estableciendo una relación entre problemática, conjunto de datos, herramientas científicas, entorno reproducible, repositorio, documentación, notebook y criterios de éxito iniciales.

En consecuencia, la Fase 1 no se evalúa únicamente por la existencia de archivos aislados, sino por la correspondencia entre lo planificado conceptualmente y lo implementado técnicamente. El avance desarrollado por el equipo materializa esta relación mediante el repositorio GitHub, el notebook F1_Definicion.ipynb, el README.md, el archivo requirements.txt, la estructura de carpetas y el registro de commits y pull requests.

Figura 11. “Mapa conceptual del flujo reproducible inicial del proyecto ABP”



Referencias APA 7 | Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. <https://git-scm.com/book/en/v2> | Project Jupyter. (2024). Jupyter Notebook Documentation. <https://docs.jupyter.org/>

Nota: El mapa conceptual organiza la relación entre problemática, datos, herramientas colaborativas, Jupyter Notebook, entorno reproducible, estructura de repositorio, documentación científica y commits planificados. Fuente: Elaboración propia del grupo (2026).

9.1 Correspondencia entre el mapa conceptual y lo implementado en F1

Tabla 9. “Correspondencia entre mapa conceptual e implementación de Fase 1”

Elemento del mapa conceptual	Implementación en Fase 1	Evidencia
Contexto de la problemática	Definición del problema asociado a degradación de datos clínicos en diabetes_raw.csv.	Informe técnico, introducción y objetivos.
Preguntas iniciales, objetivo y alcance	Formulación del problema, objetivo general, objetivos específicos y límites de Fase 1.	Capítulos iniciales del informe.
Conjunto de datos	Incorporación del archivo diabetes_raw.csv como fuente original del análisis.	Carpeta data/raw/.
Variables clave preliminares	Identificación inicial de variables clínicas y variable objetivo Outcome.	Notebook F1_Definicion.ipynb.
Flujo analítico inicial reproducible	Carga inicial del dataset, revisión de dimensiones, columnas, tipos de datos, valores faltantes, duplicados exactos y distribución de Outcome.	Notebook F1_Definicion.ipynb.
Entorno reproducible	Declaración de dependencias verificadas para ejecutar el notebook con Python 3.12.4.	Archivo requirements.txt, README.md y salida de verificación del notebook.
Jupyter Notebooks	Uso de notebook con narrativa técnica y código base.	Carpeta notebooks/ y archivo F1_Definicion.ipynb.
Documentación científica y trazabilidad	Registro de decisiones, README, informe y commits.	Informe técnico, README.md e historial GitHub.
Herramientas colaborativas	Uso de Git, GitHub, ramas y pull requests.	Historial de commits, ramas y PR cerrados/fusionados.
Estructura base del repositorio	Organización en carpetas por fase para datos, notebooks, documentación, reportes y código fuente, diferenciando componentes implementados y proyectados.	data/, docs/, notebooks/, reports/, src/.
Criterios de éxito iniciales	Repositorio creado, README preliminar, notebook inicial, dependencias y control de versiones.	Repositorio GitHub de Fase 1.

Nota: Índice de consistencia de la arquitectura del proyecto. La tabla demuestra el cumplimiento del marco conceptual propuesto, sirviendo como evidencia de que cada directriz teórica y metodológica de la planificación fue traducida en un artefacto físico, auditable y funcional dentro del repositorio.

Fuente: Elaboración propia. Integrantes Grupo 6 (2026)

La correspondencia anterior demuestra que el mapa conceptual no quedó como una planificación abstracta, sino que fue utilizado como criterio organizador para construir la arquitectura inicial del proyecto. Cada componente relevante del mapa tiene una evidencia asociada en el repositorio, en el informe o en el notebook, lo que fortalece la coherencia entre planificación y ejecución.

9.2 Elementos implementados

Durante la Fase 1 se implementaron los siguientes elementos:

- Repositorio GitHub del proyecto.
- Estructura inicial de carpetas.
- Archivo README.md preliminar.
- Archivo requirements.txt.
- Archivo .gitignore.
- Dataset diabetes_raw.csv en data/raw/.
- Notebook F1_Definicion.ipynb.
- Ramas de trabajo.
- Commits descriptivos.
- Pull requests cerrados/fusionados.
- Pull request abierto asociado a revisión documental.
- Documentación del proceso dentro del informe técnico.
- Vinculación preliminar entre mapa conceptual, repositorio y notebook.

Estos elementos representan la infraestructura mínima necesaria para continuar el desarrollo del proyecto en las siguientes fases. Su valor principal radica en que permiten mantener trazabilidad, reproducibilidad, colaboración y claridad documental desde el inicio del trabajo.

9.3 Elementos proyectados para fases posteriores

Si bien la Fase 1 establece la base técnica del proyecto, existen componentes que quedan proyectados para las fases posteriores, debido a que exceden el alcance de la implementación inicial.

Tabla 10. “Elementos proyectados para F2-F4”

Elemento proyectado	Fase estimada	Justificación
Limpieza formal de datos	F2	Corresponde al proceso ETL posterior al diagnóstico inicial.
Generación de dataset procesado	F2	Debe realizarse después de definir reglas de limpieza e imputación.
Visualizaciones exploratorias	F2	Requieren datos validados y tratamiento preliminar de inconsistencias.
Transformación de datos	F2	Normalización y tratamiento de variables categóricas, posterior a la limpieza inicial.
Funciones reutilizables en src/	F2	La modularización se desarrollará cuando el flujo de limpieza y análisis esté estabilizado.
POO	F3	Re-estructuración del notebook para soportar programación orientada a objetos y recursividad.
Modelamiento predictivo	F3	Depende de disponer de una matriz de datos procesada y validada.
Evaluación de métricas	F4	Corresponde a la validación de desempeño de modelos.
Reportes finales y gráficos	F4	Se generarán cuando existan resultados consolidados, además de generación de elementos gráficos que permitan interpretar el caso base.

Nota: Cronograma conceptual y matriz de proyección de entregables. El cuadro delimita la hoja de ruta para los próximos hitos del proyecto, justificando técnicamente la postergación de la modularización de código, la arquitectura basada en POO y el modelamiento predictivo hasta que el *pipeline* de limpieza, normalización y validación de datos esté completamente estabilizado en la Fase 2.

Fuente: Elaboración propia, Integrantes Grupo 6 (2026).

Esta delimitación es relevante porque evita extender artificialmente la Fase 1 hacia actividades que corresponden a etapas posteriores. El objetivo actual es construir una base reproducible, documentada y trazable, no resolver todavía el análisis completo del problema. Por ello, los elementos implementados y proyectados mantienen coherencia con la planificación F1-F4 y con la progresión metodológica del proyecto ABP.

9.4 Cierre de coherencia técnica

La Fase 1 permite concluir que existe una correspondencia clara entre la planificación conceptual y el avance técnico real. El mapa conceptual definió los componentes requeridos; el repositorio los materializó en una estructura organizada; el README entregó instrucciones de ejecución; el notebook incorporó el flujo computacional inicial; y GitHub registró la evolución mediante commits, ramas y pull requests.

De este modo, el avance no solo cumple una función documental, sino que establece una base técnica verificable para las siguientes etapas del proyecto. La estructura construida permite continuar hacia limpieza, análisis exploratorio, transformación, modelamiento y evaluación sin perder trazabilidad sobre las decisiones iniciales.



Capítulo X

Bibliografía

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, N., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Hastie, T., Tibshirani, R., & Friedman, J. (2017). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

Little, R. J., & Rubin, D. B. (2019). *Statistical Analysis with Missing Data* (3rd ed.). John Wiley & Sons.

Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

McKinney, W. (2010). Data structures for statistical computing in python. In *Proceedings of the 9th Python in Science Conference* (pp. 51–56). Austin, TX. <https://doi.org/10.25080/Majora-92bf1920-00a>

GitHub Docs. (s. f.). Acerca de GitHub y el control de versiones colaborativo. GitHub. <https://docs.github.com/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347–1358. <https://pubmed.ncbi.nlm.nih.gov/30943338/>

Salinas, O. (2026). *Cápsula docente: Fundamentos del ecosistema científico* [Video]. Universidad Andrés Bello. Canvas.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>

Topol, E. J. (2021). *Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again*. Basic Books.

Universidad Andrés Bello, Facultad de Ingeniería, Magíster en Ciencia de Datos e Inteligencia Artificial. (s. f.). *MCDI500: Guía de desarrollo — Sumativa 1 (Fase 1): Implementación inicial del entorno reproducible y documentación técnica* [Guía práctica con consejos para estudiantes].

Waskom, M. L. (2021). Seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>

World Health Organization. (2023). *Global strategy on digital health 2020-2025*. World Health Organization. <https://www.who.int/publications/i/item/9789240020924>